AI / ML / DL / NLP Interview Prep — Zero to Hero

System design · ML · Deep learning · NLP & LLMs · Computer vision · Statistics · Coding · MLOps · Behavioral

With "what if this happens?" scenario answers for every topic

Contents

1. 01 — System Design Fundamentals (Zero → Solid)
2. 02 — ML & LLM Systems: How They're Actually Built
3. 03 — The Debugging Framework (How to Answer "What If X Happens?")
4. 04 — 50 "What If…?" Scenarios with Model Answers
5. 05 — Case Studies: Full Interview Answers
6. 06 — Interview Playbook & Cheat Sheet
7. 07 — Machine Learning Fundamentals (Theory Round)
8. 08 — Deep Learning (Theory + Training Debugging)
9. 09 — NLP & LLMs (Theory + Scenarios)
10. 10 — Computer Vision
11. 11 — Statistics, Probability & A/B Testing
12. 12 — ML Coding Round (Implement From Scratch)
13. 13 — MLOps & Production ML (Q&A)
14. 14 — DSA Coding Round (for AI/ML roles)
15. 15 — Behavioral & Project Deep-Dive Round

AI / ML / DL / NLP Interview Prep — Zero to Hero

A complete prep course for **AI Engineer, ML Engineer, Applied Scientist, Data Scientist, NLP / CV / LLM Engineer** interviews.

📄 **Prefer one file?** Download the full course as a PDF: AI-ML-Interview-Prep.pdf

Most candidates don't fail because they lack knowledge. They fail because when the interviewer says **"What if this happens?"** they have no *structured way to think out loud*. Every file here includes those scenario questions with model answers.

Step 1 — Know which rounds you'll face

Interview round	What they test	Study file(s)
ML theory / fundamentals	bias-variance, metrics, algorithms, overfitting, imbalance	07-ml-fundamentals.md
Deep learning	backprop, optimizers, CNN/RNN/Transformers, training debugging	08-deep-learning.md
NLP / LLM	tokenization, embeddings, BERT/GPT, RLHF, LoRA, prompting, hallucination	09-nlp-and-llms.md
Computer vision	detection, segmentation, ViT, CLIP, mAP, edge deployment	10-computer-vision.md
Statistics & A/B testing	probability, Bayes, p-values, experiment design	11-statistics-and-ab-testing.md
ML coding	implement algorithms from scratch (NumPy/PyTorch)	12-ml-coding.md
DSA coding	LeetCode-style problems	14-dsa-coding-round.md
ML / LLM system design	design RAG, recommender, fraud system, LLM platform	01, 02, 05
"What if X happens?" / debugging	production failures, inconsistent outputs, drift, outages	03 (method) + 04 (50 scenarios) + "what-if" sections in 07–13
MLOps / production	CI/CD for ML, serving, monitoring, versioning	13-mlops.md
Project deep dive & behavioral	your own projects, STAR stories	15-behavioral-and-project-deep-dive.md
Interview day strategy	45-min script, phrases, cheat sheet	06-interview-playbook.md

---

Step 2 — Learn the one method for all "what if" questions

> **C**larify → **L**ayer map → **H**ypotheses (ranked) → **I**solate (how to prove it) → **F**ix (now + long-term) → **P**revent (monitoring/evals)

Read 03-debugging-framework.md **first**. It works for RAG, ML models, training bugs, data pipelines, and infrastructure.

Step 3 — Follow a plan

10-week full plan (from zero)

Week	Theory (1–1.5 h/day)	Practice (1 h/day)
1	07 ML fundamentals	14 DSA patterns 1–2
2	11 Statistics + 07 what-ifs	12 ML coding §1–5
3	08 Deep learning	14 DSA patterns 3–5
4	09 NLP & LLMs	12 ML coding §6–12
5	10 Computer vision (skip if not CV role) + 03 framework	14 DSA patterns 6–9
6	01 System design fundamentals	04 scenarios A–B (out loud)
7	02 ML & LLM systems + 13 MLOps	04 scenarios C–F
8	05 Case studies 1–4	14 DSA patterns 10–12
9	05 Case studies 5–11 + 15 project prep	2 full mock interviews
10	06 playbook, revise weak areas	4 mock interviews (mixed rounds)

2-week emergency plan (interview soon)

Days	Focus
1–2	03 framework + 04 scenarios (your domain sections)
3–4	07 + 08 Q&A (read questions, answer out loud)
5–6	09 (or 10 for CV) + 11
7–8	02 + 05 case studies 1, 4, 5
9–10	12 ML coding (top 6) + DSA patterns 1, 3, 7
11–12	15 — write 2 project deep dives + 6 STAR stories
13–14	Mocks + 06 cheat sheet

By role — what to prioritize

Role	Highest priority files
AI / LLM Engineer	03, 04, 02, 09, 05, 13, 15
ML Engineer	07, 08, 12, 13, 02, 04, 05, 14
Data Scientist / Applied Scientist	07, 11, 12, 08, 15, 05 (#4–5)
NLP Engineer	09, 08, 12, 04 (A–B), 02
CV Engineer	10, 08, 12, 13, 05 (#7–8)
MLOps Engineer	13, 02, 01, 04 (C–E), 05 (#3)

Step 4 — Practice the right way

1. **Say answers out loud.** Reading feels like learning; interviews test speaking.
2. Cover the answer → answer in 60 s (theory) or 3 min (scenario) → compare → mark misses → redo in 2 days.
3. Record mock interviews and watch them back.
4. Ask an AI or a friend to play interviewer: "*Ask me [ML theory / LLM system design / what-if] questions one by one, push back with follow-ups, then score my answer for structure, depth, and trade-offs.*"

All files

#	File
01	System design fundamentals
02	ML & LLM systems
03	Debugging framework (what-if method)
04	50 what-if scenarios
05	Case studies
06	Interview playbook & cheat sheet
07	ML fundamentals
08	Deep learning
09	NLP & LLMs
10	Computer vision
11	Statistics & A/B testing
12	ML coding from scratch
13	MLOps
14	DSA coding round
15	Behavioral & project deep dive

Rebuild the PDF

```
pip install markdown pygments
python scripts/build_pdf.py      # needs Chrome or Edge installed
```

Contributing

Found a mistake, or want to add a scenario question you were asked in a real interview? Open an issue or a pull request. New "what if" scenarios should follow the same format: **causes (ranked) → how to isolate → fix (now / long-term) → prevent.**

If this helped you, give the repo a ⭐ so others can find it.

License

Content is licensed under CC BY 4.0: free to share and adapt, with credit.

01 — System Design Fundamentals (Zero → Solid)

Even ML/AI interviews expect these basics. Every AI system is still a web system underneath.

Contents

1. The big picture
2. Networking & APIs
3. Scaling: load balancers, caching, CDN
4. Databases
5. Async processing: queues & streams
6. Distributed systems concepts
7. Reliability, monitoring, security
8. Back-of-the-envelope estimation

Part 1 — The big picture

System design = deciding which components you need and how they talk, so the system meets its requirements at scale.

Two kinds of requirements: - **Functional** — what it does ("users can upload documents and ask questions"). - **Non-functional** — how well: scale, latency, availability, consistency, cost, security.

The growth story (memorize this — it's the skeleton of every design)

```
Stage 1:  User → [Single server: app + DB]                    (works for ~100s of users)
Stage 2:  User → [App server] → [Database]                    (separate concerns)
Stage 3:  User → [Load balancer] → [App 1..N] → [Database]    (horizontal scaling)
Stage 4:  ... + [Cache] in front of DB                        (fast reads)
Stage 5:  ... + [CDN] for static files                        (global speed)
Stage 6:  ... + DB replicas (reads) + sharding (writes)       (data scale)
Stage 7:  ... + [Queue] → [Workers] for slow jobs             (async)
Stage 8:  ... + multiple regions, monitoring, autoscaling     (global + reliable)
```

The 5 qualities you trade off

Quality	Question	Typical tool
Scalability	Handle 10× users?	horizontal scaling, sharding, queues
Latency	How fast?	caching, CDN, indexes, async
Availability	Stays up when parts fail?	replication, redundancy, failover
Consistency	Everyone sees the same correct data?	single leader, transactions, quorum
Cost	Affordable?	right-sizing, caching, batch vs real-time

Part 2 — Networking & APIs

What happens when you type a URL: DNS resolves name → IP → TCP connection (+ TLS handshake for HTTPS) → HTTP request → load balancer → server → response.

Concept	One-liner
IP / DNS	IP = address of a machine; DNS = phone book from name to IP
TCP	Reliable, ordered delivery (web, APIs, DBs)
UDP	Fast, no guarantees (video calls, gaming, DNS)
HTTP/HTTPS	Request/response protocol; HTTPS = encrypted via TLS
HTTP/2, HTTP/3	Multiplexing many requests on one connection; HTTP/3 over QUIC (UDP)

API styles

Style	Use when	Example
REST	Default for public/CRUD APIs	`GET /documents/42`
GraphQL	Clients need flexible queries, many entity types	mobile apps
gRPC	Fast internal service-to-service calls (binary, typed)	model server calls
WebSocket	Two-way real-time	chat, live collaboration
SSE (server-sent events)	Server → client streaming	**streaming LLM tokens**
Long polling	Simple near-real-time fallback	notifications
Webhooks	Notify another system when something happens	"payment completed"

Good API design rules

- Nouns for resources, HTTP verbs for actions (`POST /chats/{id}/messages`).
- **Pagination** (cursor-based for large lists), **versioning** (`/v1/`), **idempotency keys** for POST that must not run twice (payments), proper status codes (200, 201, 400, 401, 403, 404, 429, 500, 503).
- **Rate limiting** per user/API key.

Part 3 — Scaling: load balancers, caching, CDN

Vertical vs horizontal scaling

- **Vertical** = bigger machine. Simple, but has a ceiling and is a single point of failure.
- **Horizontal** = more machines. Needs **stateless** servers (sessions in Redis/DB, not memory).

Load balancer

Spreads requests across servers, removes unhealthy ones (health checks). - **L4** (TCP level, fast) vs **L7** (HTTP level — can route by URL/header, e.g., `/api/llm` → GPU pool). - Algorithms: round-robin, least

connections, weighted, consistent hashing (sticky by key). - Related: **reverse proxy** (Nginx), **API gateway** (auth, rate limiting, routing, logging in one place).

Caching

Store results of expensive operations in fast memory (Redis/Memcached).

Pattern	How it works	Use
Cache-aside (most common)	App checks cache → miss → read DB → write cache	general reads
Write-through	Write to cache and DB together	data read right after write
Write-back	Write to cache, flush to DB later	high write rate, risk of loss
TTL	Entries expire after time	everything

- **Eviction:** LRU (least recently used) is the default answer.
- **Problems you should mention:**
- *Stale data* → TTL + invalidate on write.
- *Thundering herd / cache stampede* → many requests miss at once and hit DB → use locks / request coalescing / staggered TTLs.
- *Hot key* → one key gets huge traffic → replicate key, local cache.
- **AI-specific:** exact-match cache for prompts, **semantic cache** (embed query, return cached answer if similarity > threshold), embedding cache, **KV/prefix cache** in LLM servers.

CDN

Copies static files (images, JS, videos, model files) to servers worldwide. Users download from nearest location.

Rate limiting algorithms

- **Token bucket** (most common): bucket refills at rate R, each request takes a token; allows bursts.
- **Leaky bucket**: smooth constant output.
- **Fixed window / sliding window counter**: count requests per time window.
- Store counters in Redis for distributed limits. For LLMs, limit **tokens per minute**, not just requests.

Part 4 — Databases

SQL vs NoSQL

	SQL (Postgres, MySQL)	NoSQL
Model	Tables, relations, joins	Key-value (Redis, DynamoDB), document (MongoDB), wide-column (Cassandra), graph (Neo4j)
Strength	Transactions (ACID), complex queries, correctness	Massive scale, flexible schema, high write throughput

	SQL (Postgres, MySQL)	NoSQL
Pick when	Payments, users, orders, metadata	Chat messages, event logs, sessions, feeds

Interview default: "I'll start with Postgres for metadata because we need consistency and joins; I'd move high-volume append-only data like chat messages to Cassandra/DynamoDB."

Specialized stores you should name: - **Object storage** (S3/GCS): files, images, model weights, datasets — cheap, huge. - **Vector DB** (Pinecone, Weaviate, Milvus, Qdrant, pgvector): embeddings + similarity search. - **Search engine** (Elasticsearch/OpenSearch): full-text keyword search, inverted index, BM25. - **Data warehouse / lakehouse** (BigQuery, Snowflake, Databricks): analytics, training data. - **Time-series DB**: metrics. **Feature store** (Feast, Tecton): ML features.

ACID vs BASE

- **ACID** — Atomic, Consistent, Isolated, Durable (bank transfers).
- **BASE** — Basically Available, Soft state, Eventually consistent (likes counter).

Indexes

Like a book index — makes reads fast, writes slightly slower. B-tree (most SQL), LSM-tree (write-heavy: Cassandra, RocksDB). For vectors: **HNSW** (graph, fast, memory-heavy), **IVF** (clusters), **PQ** (compression).

Replication

Copies of data on multiple machines. - **Leader-follower:** writes go to leader, reads from followers. Followers may lag → **replication lag** (user writes, then reads old data). Fix: read-your-own-writes from leader. - **Multi-leader:** writes in multiple regions; must resolve conflicts. - **Leaderless (quorum):** write to W nodes, read from R nodes; if **W + R > N** you read latest data.

Sharding (partitioning)

Split data across machines. - **By hash of key** (even spread) — `shard = hash(user_id) % N`. - **By range** (good for range queries, risk of hot spots). - **By tenant** (common in B2B/RAG — isolation per customer). - **Consistent hashing:** keys and servers on a ring; adding a server only moves ~1/N keys. (Used in caches, Cassandra, DynamoDB.) - Problems: hot shards, cross-shard joins, resharding.

Denormalization

Duplicate data to avoid joins for fast reads (e.g., store author name inside post). Trade-off: harder updates.

Part 5 — Async processing: queues & streams

Why: don't make users wait for slow work (embedding a 500-page PDF, sending emails, training jobs).

```
API → [Queue] → Workers (scale independently) → result in DB → notify user (webhook / polling / websocket)
```

Tool	Type	Use
RabbitMQ / SQS	Message queue (task consumed once)	background jobs
Kafka / Kinesis / Pub/Sub	Log/stream (many consumers, replayable)	event streaming, feature pipelines, logs

- **Delivery guarantees:** at-most-once, **at-least-once (default)**, exactly-once (hard). With at-least-once, make consumers **idempotent**.
- **Dead-letter queue (DLQ):** messages that fail repeatedly go aside for inspection.
- **Backpressure:** when consumers can't keep up — queue grows; autoscale workers or slow producers.
- **Batch vs stream processing:** Spark (batch, hourly/daily) vs Flink/Kafka Streams (real-time). ML: batch features vs real-time features.

Architecture styles

- **Monolith** — one deployable. Start here. Simple.
- **Microservices** — separate services per domain; independent scaling & deploys; cost: network calls, distributed debugging.
- **Event-driven** — services react to events on Kafka.
- **CQRS** — separate write model and read model.
- **Saga** — distributed transaction as a sequence of local steps with compensating actions.

Part 6 — Distributed systems concepts

Concept	Plain explanation
CAP theorem	During a network **P**artition you must choose **C**onsistency (reject requests) or **A**vailability (serve possibly stale data). Payments → CP. Social feed → AP.
PACELC	Even without partition: trade-off between **L**atency and **C**onsistency.
Eventual consistency	All replicas converge eventually; reads may be stale briefly.
Strong consistency	Every read sees latest write. Slower.
Consensus (Raft/Paxos)	Nodes agree on a value/leader despite failures (etcd, ZooKeeper).
Leader election	Pick one node to coordinate (e.g., one scheduler for training jobs).
Idempotency	Doing an operation twice = same result as once. Essential with retries.
Retries + exponential backoff + jitter	Retry after 1s, 2s, 4s (+ random) to avoid overwhelming a recovering service.
Circuit breaker	Stop calling a failing service for a while; return fallback.
Bulkhead	Isolate resources so one failure doesn't sink everything (separate pools).
Timeouts	Every network call needs one.
Unique IDs	UUID (random), Snowflake (time + machine + sequence, sortable).
Bloom filter	Tiny structure answering "definitely not in set / maybe in set" (crawler seen-URLs).
Heartbeats / gossip	Detect dead nodes / spread cluster state.

Concept	Plain explanation
2PC	Two-phase commit across DBs — consistent but slow and blocking.

Part 7 — Reliability, monitoring, security

Availability numbers

Availability	Downtime/year
99%	3.65 days
99.9%	8.8 hours
99.99%	52 minutes
99.999%	5 minutes

- **No single point of failure:** redundancy everywhere (multiple servers, replicas, zones).
- **Failover:** active-passive or active-active.
- **SLI / SLO / SLA:** indicator (p95 latency), objective (p95 < 2s for 99% of days), agreement (contract with penalties).

Observability (3 pillars)

- **Metrics** — numbers over time (QPS, latency p50/p95/p99, error rate, GPU utilization, token usage).
- **Logs** — events with details.
- **Traces** — one request followed across all services (OpenTelemetry). **For LLM apps: trace every step — retrieval, prompt, model call, tokens, cost.**

Deployments

- **Blue-green:** two environments, switch traffic.
- **Canary:** send 1–5% of traffic to new version, watch metrics, then expand.
- **Shadow:** new version receives copy of traffic, its outputs are not shown to users (great for ML models).
- **Feature flags:** turn features on/off without deploy.

Security basics

- **AuthN** (who are you: OAuth2/OIDC, JWT) vs **AuthZ** (what can you do: RBAC/ABAC).
- Encryption in transit (TLS) and at rest.
- Secrets in a vault, never in code/prompts.
- Least privilege, audit logs, input validation, multi-tenant isolation.

Part 8 — Back-of-the-envelope estimation

Numbers to memorize

1 day	≈ 86,400 s ≈ **10^5 s**
1M requests/day	≈ **12 QPS**
100M requests/day	≈ 1,200 QPS
Peak traffic	≈ 2–3× average
Memory read	~100 ns
SSD read	~100 µs
Same-datacenter network round trip	~0.5 ms
Cross-continent round trip	~150 ms
Redis get	~1 ms
DB indexed query	~5–50 ms
LLM first token	~200–800 ms
LLM generation	~30–100 tokens/s per stream
1 token	≈ 0.75 English words ≈ 4 chars
Embedding (float32, 1536 dims)	1536 × 4 B ≈ **6 KB**
7B model in FP16	≈ 14 GB GPU memory (2 bytes/param)

Formula

```
QPS          = daily_requests / 86,400          (× 2–3 for peak)
Storage      = items × size × retention
Bandwidth    = QPS × response_size
Servers      = peak QPS / QPS_per_server
```

Worked example — RAG for a company with 10,000 employees

- Each asks 10 questions/day → 100k/day → **~1.2 QPS avg, ~5 QPS peak**. (Small! Don't over-engineer.)
- Corpus: 1M documents × 10 chunks = **10M chunks** × 6 KB = **60 GB vectors** (+ HNSW overhead ~1.5×) → fits on one vector DB node with replicas; or compress with PQ.
- LLM tokens: 100k × (3k input + 300 output) = 330M tokens/day → cost estimate from price per million tokens.
- Conclusion: the bottleneck is **LLM cost and latency**, not QPS → focus on caching, routing, context size.

> **Interview tip:** Always say what the numbers *mean* for the design ("5 QPS means a single service with 2 replicas for HA is enough; the real concern is cost").

02 — ML & LLM Systems: How They're Actually Built

The interviewer for an AI/ML role wants to see you think **beyond the model**: data, serving, evaluation, monitoring, cost.

> **Golden truth:** In production ML, the model is ~10% of the system. The other 90% is data pipelines, serving, monitoring and feedback loops.

Contents

1. The ML lifecycle
2. Data & features
3. Training infrastructure
4. Model serving
5. Deployment & experimentation
6. Monitoring ML systems
7. LLM systems: RAG in depth
8. LLM serving & optimization
9. Agents & tool use
10. Evaluation of LLM systems
11. Safety, security & governance
12. Decision tables (when to use what)

---

1. The ML lifecycle

```
Problem framing → Data collection → Labeling → Feature engineering → Training → Evaluation
       ↑                                                                          │
       └── Monitoring ← Serving ← Deployment (canary/shadow) ← Model registry ←──┘
                 │
                 └── feedback / new labels → retraining
```

Problem framing questions (ask these in every ML design interview): - What is the business goal? What is the ML objective (proxy)? Are they aligned? - Is ML even needed? (Rules/heuristics baseline first.) - Online (real-time) or batch prediction? Latency budget? - Where do labels come from? How delayed are they? - What's the cost of a false positive vs false negative?

2. Data & features

- **Feature store** = one place to define features, used by both training (offline store, historical, point-in-time correct) and serving (online store, low latency, e.g., Redis). **Main reason: prevents train/serve skew.**

- **Point-in-time correctness:** when building training rows, only use feature values that existed *at that time* — otherwise leakage.
- **Batch features** (computed daily: "user's purchases last 30 days") vs **real-time features** (streaming: "transactions in last 5 minutes").
- **Data validation:** schema, null rates, ranges, distribution checks before training.
- **Data versioning:** DVC, Delta Lake/Iceberg time travel.
- **Labeling:** human labelers, weak supervision, implicit feedback (clicks), LLM-assisted labeling + human review.

3. Training infrastructure

- Orchestration: Airflow / Kubeflow / Prefect / SageMaker pipelines.
- Experiment tracking: MLflow, Weights & Biases.
- Distributed training: **data parallel** (copy model, split data — DDP), **model/tensor parallel** (split model across GPUs), **pipeline parallel**, **FSDP/ZeRO** (shard optimizer state + params). Mixed precision (bf16).
- Checkpointing (resume after spot instance preemption).
- **Model registry:** versioned artifacts + metadata + stage (staging/production).

4. Model serving

Mode	When	Example
Batch	Predictions can be precomputed	nightly recommendations, churn scores
Online (real-time)	Need fresh prediction per request	fraud check at payment
Streaming	Predict on events as they arrive	anomaly detection on logs
On-device / edge	Privacy, offline, low latency	keyboard prediction

Serving stack: model server (Triton, TorchServe, TF Serving, BentoML, vLLM for LLMs) behind an API, autoscaling, **dynamic batching** (group requests to use GPU efficiently), model caching. Optimization: quantization, distillation, pruning, ONNX/TensorRT compilation.

Two-stage pattern (search/recs/ads): cheap **candidate generation** (millions → 1000s, e.g., ANN on embeddings) → expensive **ranking** model (1000s → 100) → **re-ranking** with business rules (diversity, freshness).

5. Deployment & experimentation

- **Shadow mode** → **canary** → **A/B test** → full rollout. Keep previous model warm for instant rollback.
- **A/B testing:** randomize by user, pick primary metric + guardrail metrics (latency, errors, revenue), power analysis for sample size, beware novelty effect and peeking.
- **Multi-armed bandits** when you want to explore + exploit continuously.
- **Champion / challenger** for automated retraining.

6. Monitoring ML systems

Layer	What to monitor
System	latency, throughput, errors, GPU/CPU/memory
Data	missing values, schema, feature distributions (**data drift**: PSI, KS test)
Model	prediction distribution, confidence, **performance when labels arrive** (concept drift)
Business	conversion, fraud losses, user satisfaction
Alert → investigate → retrain/rollback. Retraining triggers: schedule, drift threshold, performance drop.	

---

7. LLM systems: RAG in depth

RAG (Retrieval-Augmented Generation) = fetch relevant knowledge, put it in the prompt, let the LLM answer grounded in it. Use RAG when knowledge changes, is private, or needs citations. (Fine-tuning teaches *style/format/behavior*, not fresh facts.)

Offline: ingestion pipeline

```
Sources (PDF, Confluence, DB, web) → Connectors → Parse (OCR, tables, layout)
 → Clean (remove boilerplate, dedupe) → Chunk → Enrich metadata (title, section, date, ACL,
tenant)
 → Embed → Upsert into vector index (+ keyword index) → Version the index
```

Chunking choices: | Strategy | Notes | |----------|-------| | Fixed size (e.g., 500 tokens, 50 overlap) | Simple baseline | | Recursive / by structure (headings, paragraphs) | Better coherence — usually best default | | Semantic chunking | Split where topic changes | | Parent-child / small-to-big | Retrieve small precise chunk, send bigger parent section to LLM | | Per table row / per FAQ item | Structured content | Add **contextual headers** to each chunk: "Doc: HR Policy 2025 > Section: Leave > ..." — improves retrieval a lot.

Online: query pipeline

```
Query → guardrails → (rewrite / expand / decompose / HyDE) → embed
 → hybrid retrieval (vector + BM25) with metadata/ACL pre-filter → merge (RRF)
 → rerank (cross-encoder) → top-k → build context (order, dedupe, token budget)
 → prompt (instructions + context + question + cite sources) → LLM → output guardrails +
citation check → stream to user
```

Key techniques to name: - **Hybrid search** — vectors capture meaning, BM25 captures exact words (product codes, names). Combine with **Reciprocal Rank Fusion**. - **Reranking** — cross-encoder scores (query, chunk) pairs jointly; big precision boost. - **Query rewriting** — turn "what about for contractors?" into a standalone query using chat history. - **HyDE** — generate a hypothetical answer, embed it, search with that. - **Multi-query** — generate several phrasings, union results. - **Metadata filtering** — tenant,

permissions, date, doc type. Pre-filter for security. - **GraphRAG / knowledge graphs** — for entity-relationship and multi-hop questions. - **Agentic RAG** — LLM decides when/what to retrieve iteratively.

Vector DB essentials

- Similarity: cosine / dot product / L2 (normalize vectors for cosine).
- ANN indexes: **HNSW** (high recall, fast, RAM heavy), **IVF** (cluster then search nearby clusters), **PQ** (compress vectors, less memory, lower recall). ANN = *approximate* → results can vary slightly → relevant to consistency questions!
- Scaling: sharding, replicas, namespaces per tenant, quantization.
- Choice: pgvector (already on Postgres, < ~10M vectors), managed (Pinecone), self-hosted (Milvus, Qdrant, Weaviate), Elasticsearch/OpenSearch (if you need hybrid in one system).

8. LLM serving & optimization

Technique	What it does
Streaming (SSE)	Show tokens as generated — lowers perceived latency
Continuous batching (vLLM, TGI)	New requests join the batch mid-generation → much higher GPU throughput
PagedAttention / KV cache	Efficient memory for attention cache
Prefix / prompt caching	Reuse computation for repeated system prompts/context
Quantization (INT8/INT4, AWQ, GPTQ)	Smaller memory, faster, slight quality loss
Speculative decoding	Small draft model proposes tokens, big model verifies
Distillation	Train small model to mimic big one
Model routing / cascade	Cheap model first; escalate hard queries to big model
Semantic cache	Return cached answer for similar question
LLM gateway	One internal API: auth, rate limits, quotas per team, fallback providers, logging, cost tracking

Key metrics: TTFT (time to first token), TPOT/ITL (time per output token), throughput (tokens/s), cost per 1k requests. **Memory rule of thumb:** params × bytes (FP16 = 2 B) + KV cache. 70B FP16 ≈ 140 GB → multiple GPUs (tensor parallel) or quantize.

API vs self-hosted: | API (closed models) | Self-hosted (open models) | |---|---| | Best quality, zero infra, fast start | Data control, cost at high volume, customizable | | Per-token cost, rate limits, vendor changes | GPU ops burden, scaling, lower top-end quality |

9. Agents & tool use

Agent loop: **LLM plans → calls a tool → observes result → repeats → final answer.** Design concerns: tool schemas & descriptions, max steps/cost limits, state/memory, error handling, **human-in-the-loop for risky actions**, sandboxing code execution, observability of each step, evaluation of trajectories (did it pick

the right tools?), permissions per tool (least privilege). Patterns: ReAct, plan-and-execute, router + specialist agents, multi-agent (use only when justified — more agents = more failure points).

10. Evaluation of LLM systems

Level	Metrics / methods
Retrieval	recall@k, precision@k, MRR, nDCG (needs labeled query→doc pairs)
Generation	faithfulness/groundedness, answer relevance, correctness vs reference, citation accuracy
Consistency	agreement rate across repeated runs / paraphrased questions
Safety	jailbreak success rate, toxicity, PII leakage
Online	thumbs up/down, resolution rate, escalation rate, retention, latency, cost
Methods: golden datasets (built from real queries), LLM-as-judge (with rubric, validated vs humans), human review of samples, A/B tests, **regression evals in CI on every prompt/model change**.	

11. Safety, security & governance

- Input guardrails: prompt-injection detection, topic restriction, PII detection.
- Output guardrails: toxicity, PII, policy, schema validation, groundedness.
- **Indirect prompt injection** via retrieved docs/web/email — treat all retrieved content as untrusted.
- Multi-tenant isolation (separate namespaces, ACL pre-filter, scoped caches).
- Data retention, audit logs, model cards, compliance (GDPR right-to-be-forgotten → must delete from index **and** caches **and** logs; hard for fine-tuned weights → another reason to prefer RAG).

12. Decision tables

Prompting vs RAG vs fine-tuning | Need | Use | |------|-----| | Fresh/private facts, citations | **RAG** | | Specific output style/format, domain jargon, smaller cheaper model | **Fine-tuning** (often LoRA) | | Quick iteration, general task | **Prompt engineering** first, always | | Both knowledge + behavior | RAG + fine-tuning |

Real-time vs batch prediction | Real-time | Batch | |-----------|-------| | Depends on request-time context (fraud at checkout) | Inputs known in advance (daily recs) | | Harder: latency, online features | Easier, cheaper, can use big models |

Which metric? | Problem | Metric | |---------|--------| | Imbalanced classification | precision/recall, PR-AUC, F1 | | Ranking | nDCG, MRR, recall@k | | Regression | MAE / RMSE | | Business | revenue, retention, CTR (with guardrails) |

03 — The Debugging Framework (How to Answer "What If X Happens?")

This is the most important file. Interviewers ask scenario questions because they want to see **how you think**, not whether you memorized the answer. A calm, structured answer with an imperfect conclusion beats a list of random guesses every time.

The framework: C.L.H.I.F.P.

> **C**larify → **L**ayer map → **H**ypotheses → **I**solate (measure) → **F**ix (now + long-term) → **P**revent

Step	What you do	What you say
1. Clarify	Pin down the symptom. Who, when, how often, since when, what changed?	"Before I jump in — is this for *all* users or some? Started suddenly or gradually? Did we deploy anything recently?"
2. Layer map	Draw the request path. Every layer is a suspect.	"Let me walk the request path: query → rewrite → embed → retrieve → rerank → prompt → LLM → post-process."
3. Hypotheses	List causes per layer, **ranked by likelihood**.	"The most likely causes, in order, are: sampling temperature, retrieval non-determinism, and index version mismatch."
4. Isolate	How you'd *prove* which one it is, cheapest test first.	"I'd replay the same query 20 times and log retrieved chunk IDs. If chunks are identical but answers differ → it's generation. If chunks differ → it's retrieval."
5. Fix	Short-term mitigation + long-term fix.	"Short-term: temperature 0 and pin the model version. Long-term: versioned index with atomic swaps."
6. Prevent	Monitoring, evals, alerts, tests so it doesn't recur.	"I'd add a consistency eval to CI and alert when answer agreement drops below 90%."

Memory trick: "*Can Large Hippos Itch For Pizza?*"

Step 2 in detail — the layer maps

You must be able to draw these from memory. When asked any scenario, **draw the relevant one first**.

RAG / LLM application

```
User
 │
 ▼
[API gateway / auth / rate limit]
 │
```

```
 ▼
[Conversation memory]  ← chat history, user profile
 │
 ▼
[Query understanding]  ← rewrite, expand, classify intent, guardrails (input)
 │
 ▼
[Embedding model]      ← query → vector
 │
 ▼
[Retriever]            ← vector DB (ANN) + keyword (BM25) + metadata/ACL filters
 │
 ▼
[Reranker]             ← cross-encoder, top-k selection
 │
 ▼
[Context builder]      ← order chunks, truncate to token budget, add citations
 │
 ▼
[Prompt template]      ← system prompt + context + question
 │
 ▼
[LLM]                  ← model version, temperature, max tokens, provider
 │
 ▼
[Post-processing]      ← guardrails (output), format, citation check
 │
 ▼
[Cache]  [Logs/traces]  [Feedback]
```

Offline side (where many bugs live):

```
Source docs → parse/OCR → clean → chunk → embed → upsert to vector index → (index version)
```

Classic ML model in production

```
Raw data → ETL → Feature pipeline → Feature store (offline + online)
                                          │                  │
                                    Training job       Serving API ← request
                                          │                  │
                                    Model registry ──→ Model server → prediction → logging →
monitoring
```

Generic web service

```
Client → DNS → CDN → Load balancer → App servers → Cache → Database
                                          │
                                       Queue → Workers
```

Step 3 in detail — universal causes checklist

When you're stuck, run through these. One of them is almost always the answer.

Category	Questions to ask yourself
Change	What was deployed / changed? Code, model, prompt, data, config, dependency, provider?
Data	Did the input data change (drift, schema, nulls, new source, stale)?
Randomness	Is anything non-deterministic (sampling, ANN search, GPU math, random seeds, ties)?
State	Is there per-user state (history, cache, personalization, permissions)?
Versions	Can different replicas serve different versions (model, index, prompt, A/B test)?
Capacity	Is something overloaded (timeouts, rate limits, queue backlog, memory, GPU)?
Dependencies	Is a downstream service slow/failing (vector DB, LLM provider, DB)?
Train/serve skew	Is production computing something differently than training/evaluation did?
Feedback loops	Is the model influencing its own future training data?

Step 4 in detail — isolation techniques

Technique	When to use
Replay the same input N times, log every intermediate output	Inconsistency, non-determinism
Diff a good request vs. bad request trace	"Works for some users, not others"
Bisect — freeze all layers except one	Any multi-layer pipeline
Timeline correlation — overlay metric drop with deploy/data events	Sudden degradation
Slice metrics by segment (region, device, user type, doc type, language)	"Accuracy dropped"
Golden set — fixed eval questions with known answers	Quality regressions
Shadow / canary — run new version alongside old	Before shipping a fix

Golden sentence for interviews:

> "I'd find the *first layer where the output differs* between the good and bad case — that's where the bug lives."

Step 5 — always give TWO fixes

Interviewers love this distinction:

- **Mitigation (stop the bleeding, minutes–hours):** rollback, feature flag off, fallback model, cache, rate limit, raise timeout, serve "I don't know".
- **Root-cause fix (days–weeks):** re-architecture, retraining, new pipeline, versioning, data contracts.

Step 6 — prevention vocabulary

Use these words: **evals in CI**, **golden datasets**, **canary deploys**, **shadow traffic**, **drift monitoring**, **SLOs + alerts**, **tracing (request ID through every layer)**, **data contracts / schema validation**, **versioning (model, prompt, index, data)**, **runbooks**, **post-mortem**.

Full worked example — the question you were asked

> **"Your RAG app has the same information, but gives different users different answers — and sometimes no answer at all. What do you do?"**

Speak it like this (≈4 minutes)

1. Clarify (30 sec)

> "A few questions first. Is it the *exact same question text*, or similar questions? Different answers in wording, or in facts? Does the same user get different answers when they retry, or only different users? And 'no answer' — is that an 'I don't know', an error, or a timeout? Did anything change recently — model, prompt, documents?"

(Interviewer will give you some answers. If they say "you decide", state your assumptions.)

2. Layer map (30 sec) — draw the RAG pipeline above.

3. Hypotheses, ranked (1 min)

Different answers to the same question: 1. **Generation randomness** — temperature > 0, no seed. Most common cause. 2. **Per-user context** — chat history, user profile, or name/date injected into the prompt makes prompts differ. 3. **Permission / metadata filters** — users have different access rights (ACLs), so they retrieve different documents. *This may be correct behavior!* 4. **Retrieval non-determinism** — approximate nearest-neighbour search, ties in scores, different replica returns slightly different top-k, chunk order changes. 5. **Version skew** — replicas have different index versions (mid-reindex) or different model/prompt versions (A/B test, canary, provider silently updated model). 6. **Conflicting source docs** — two documents say different things (old policy vs new); whichever gets retrieved wins. 7. **Query rewriting** — an LLM rewrites the query non-deterministically, so retrieval differs. 8. **Caching** — some users hit a stale cached answer, others get a fresh one.

Sometimes no answer: 1. **Retrieval returns nothing above the similarity threshold** → model correctly says "I don't know" (threshold too strict, or query phrasing differs from document vocabulary). 2. **Context overflow / truncation** — long chat history pushes the retrieved chunks out of the token budget. 3. **Guardrail false positives** — input/output filters block legitimate answers. 4. **Timeouts / rate limits** from vector DB or LLM provider → app swallows the error and returns empty. 5. **Chunking bug** — the answer is split across chunks, or the doc failed to parse (PDF table, scanned image) and never got indexed.

4. Isolate (1 min)

> "I'd pick a few failing questions and **replay each one 20 times**, logging a full trace: rewritten query, retrieved chunk IDs and scores, final prompt hash, model version, and the answer. - If **chunk IDs differ** between runs → the problem is in retrieval (non-deterministic ANN, index version, query rewrite). - If **chunk IDs are the same but answers differ** → it's generation (temperature, prompt differences). - If **the prompt hashes differ** for different users → it's per-user context (history, profile, ACLs). - For 'no answer' cases I'd check: did retrieval return 0 chunks? Was there an error or timeout? Did a guardrail fire?"

5. Fix (1 min)

Cause	Fix
Sampling	temperature = 0 (or low), fixed seed where supported, pin exact model version (`model-2025-xx-xx`, not `latest`)
Retrieval non-determinism	deterministic tie-break (sort by score, then doc_id); exact search for small corpora; hybrid search (BM25 + vectors) for stability
Version skew	**versioned indexes** + atomic alias swap (build new index fully, then flip pointer); pin model/prompt version per request and log it
Conflicting docs	dedupe, add `effective_date` / `source_of_truth` metadata, prefer latest, remove superseded docs
Per-user context	limit/ summarize history; keep volatile data (timestamp) out of prompt unless needed
ACLs	that's expected — but make the answer say "based on documents you have access to"
No retrieval hits	tune threshold on an eval set; query expansion / HyDE; hybrid search; better chunking
Overflow	token budgeting: reserve fixed space for retrieved context; summarize history
Errors	retries with backoff, fallback model, explicit error messages instead of empty answers
Cache	invalidate cache on index update (include index version in cache key)

6. Prevent (30 sec)

> "I'd add a **consistency eval**: run each golden question 5–10 times and measure answer agreement — alert if below threshold. Add **retrieval metrics** (recall@k on a labeled set) and **groundedness** checks to CI. Log the model, prompt and index version on every request so any answer is reproducible. And monitor the rate of 'no answer' responses per day — a spike tells me retrieval or the provider broke."

Why this answer wins: it shows you know the full pipeline, you rank causes, you know how to *prove* the cause, and you think about production.

Phrases that buy you thinking time (and sound senior)

- "Let me first make sure I understand the symptom…"
- "Let me walk through the request path, because every layer is a suspect."
- "The cheapest thing to check first is…"
- "I'd want to separate *retrieval* problems from *generation* problems."
- "This could actually be correct behavior if… — so I'd confirm the expected behavior first."
- "Short-term I'd mitigate by…, and the long-term fix is…"
- "To make sure this never happens silently again, I'd monitor…"
- "The trade-off here is… (latency vs quality / cost vs accuracy / freshness vs consistency)."

What NOT to do

- ❌ Jump straight to "I'd fine-tune the model." (Most expensive fix, rarely the cause.)
- ❌ List 15 causes with no order. Rank them.
- ❌ Say "I'd check the logs" without saying **what** you'd look for in them.
- ❌ Forget the short-term mitigation. Production is burning *now*.
- ❌ Stay silent while thinking. Think out loud.

04 — 50 "What If…?" Scenarios with Model Answers

Every answer follows the framework from 03-debugging-framework.md: **Causes (ranked) → How to isolate → Fix (now / long-term) → Prevent.**

How to practice: read only the question, cover the rest, speak for 3 minutes, then compare. Mark the ones you missed and redo them 2 days later.

Contents

- A. RAG systems (1–12)
- B. LLM apps & agents (13–22)
- C. ML models in production (23–32)
- D. Data & pipelines (33–38)
- E. Serving, infra & scale (39–46)
- F. Recommenders, search & ranking (47–50)

A. RAG systems

1. Same question, different answers for different users (or on retry)

Full worked answer in 03-debugging-framework.md. **Short version:** temperature/sampling → per-user prompt differences (history, ACLs) → non-deterministic retrieval (ANN, ties, query rewrite) → version skew (index/model/prompt) → conflicting docs → cache. **Isolate:** replay N times, log chunk IDs + prompt hash + model version. Same chunks + different answers = generation. Different chunks = retrieval. **Fix:** temp 0 + pinned model, deterministic tie-breaks, versioned index with atomic swap, dedupe conflicting docs, version in cache key. **Prevent:** consistency eval (agreement over N runs), log versions per request.

2. RAG says "I don't know" even though the answer is in the documents

Causes (ranked): 1. Document never indexed properly — PDF parse failure, scanned image without OCR, table flattened into garbage. 2. Chunking split the answer across chunks, or chunk too big so the embedding is diluted. 3. Vocabulary mismatch — user says "PTO", doc says "paid leave". Pure vector search misses exact terms, codes, IDs. 4. Similarity threshold too strict / top-k too small. 5. Metadata filter wrong (wrong tenant, date filter, ACL). 6. Answer retrieved but ranked low → cut off by reranker or context budget. 7. Prompt too strict ("only answer if 100% sure").

Isolate: search the vector DB directly for the doc → is it there? Inspect the chunk text. Run the query and print top-50 with scores — where does the right chunk rank? If not in top-50 → embedding/chunking. In top-50 but not top-5 → reranking/k. **Fix:** better parser/OCR, chunk with overlap & respect headings, **hybrid search (BM25 + vector)**, query expansion/rewrite, raise k then rerank, tune threshold on labeled set. **Prevent:** retrieval eval set with **recall@k**, ingestion validation (empty-chunk checks, parse-success rate).

3. RAG gives confident wrong answers (hallucinations)

Causes: retrieved chunks are irrelevant but model answers anyway; model uses its own pretrained knowledge; outdated docs; prompt doesn't require grounding; conflicting chunks. **Isolate:** check if the answer text is supported by retrieved chunks (groundedness / faithfulness check). Classify failures: *retrieval failure* (right doc not retrieved) vs *generation failure* (right doc retrieved, model ignored it). **Fix:** prompt "answer only from context, cite sources, say I don't know otherwise"; require citations and verify cited chunk contains the claim; reranker + relevance threshold; smaller context with higher precision; post-hoc NLI/LLM-judge faithfulness check before returning. **Prevent:** faithfulness metric in evals (e.g., RAGAS-style), user feedback thumbs-down loop, hallucination rate dashboard.

4. Answers are outdated after the source docs were updated

Causes: ingestion pipeline not triggered / failing silently; old chunks not deleted (upsert adds new, old stays → both retrieved); answer cache not invalidated; embedding batch job lags. **Isolate:** compare doc `last_modified` vs. index `ingested_at`; search index for both old and new versions of the doc. **Fix:** change-data-capture (event on doc update) → re-ingest; **delete-by-doc_id then insert** (idempotent upserts keyed by doc_id + chunk_no); cache key includes index version; prefer newest via `effective_date` metadata. **Prevent:** freshness SLO ("95% of doc updates searchable in < 15 min"), alert on ingestion lag.

5. Retrieval quality dropped after switching embedding model

Cause: almost always — **query embedded with new model, documents still embedded with old model** (different vector spaces) or partially re-embedded index. Also: different normalization (cosine vs dot product), different max token length truncating chunks. **Fix:** re-embed entire corpus into a **new index**, test offline, then swap atomically. Never mix models in one index. Store `embedding_model_version` as index metadata and assert at query time. **Prevent:** retrieval eval gate before swap.

6. RAG is too slow (p95 = 8 s)

Isolate first: trace per stage. Typical budget: embed 20 ms, retrieve 30–100 ms, rerank 100–300 ms, LLM first token 300–800 ms, full generation 1–5 s. **Fixes by stage:** - LLM: **stream tokens** (perceived latency), smaller/faster model for simple queries (routing), shorter context, limit output tokens, prompt caching, speculative decoding/self-hosted vLLM. - Retrieval: ANN index (HNSW) tuning, fewer filters post-hoc, colocate DB. - Rerank: rerank top-20 not top-100, smaller cross-encoder. - Query rewrite: skip it for short clear queries, or run in parallel. - Cache: exact-match + **semantic cache** for frequent questions. - Parallelize independent calls (retrieve from 2 sources concurrently). **Trade-off:** latency vs quality — measure quality impact of each cut on eval set.

7. Users see documents they shouldn't have access to

Severity: security incident — say so. Mitigate immediately (disable feature or restrict to safe corpus). **Causes:** ACL filtering done *after* retrieval in the app (bug) or not at all; ACL metadata stale (permission revoked but index not updated); shared cache across users returns another user's answer; prompt injection asking to "show all docs". **Fix:** enforce **pre-retrieval filtering** in the vector DB with tenant/group IDs; per-tenant namespaces/indexes for strong isolation; cache key includes user/permission scope; sync ACL changes on events. **Prevent:** automated tests "user A queries doc B → must not appear", audit logs.

8. Works great in demo, bad in production

Causes: eval set doesn't represent real queries (demo questions were written by the team who know the docs); real users write short, vague, misspelled, multi-language queries; production corpus 100× bigger → more distractors. **Fix:** sample real production queries → label → new eval set; add query rewriting/clarifying questions; hybrid search; reranker; slice metrics by query type. **Prevent:** continuous eval on sampled production traffic.

9. Answer quality is fine for short docs, bad for long docs (100+ page manuals)

Causes: fixed-size chunks lose structure; tables/sections split; "lost in the middle" — LLMs ignore middle of long context. **Fix:** structure-aware chunking (by headings), parent-child retrieval (retrieve small chunk, give parent section), add doc title/section path to each chunk, summarise sections, put most relevant chunks at start/end of context.

10. Multi-hop questions fail ("Which manager approved the project that had the biggest budget?")

Cause: single retrieval can't fetch both facts. **Fix:** query decomposition (LLM splits into sub-questions), iterative/agentic retrieval, knowledge graph for entity relations, or SQL tool for structured data. **Trade-off:** more LLM calls = more latency and cost.

11. Cost of RAG app exploded this month

Isolate: cost per request × requests; break down tokens: input (context!) vs output; which feature/user/tenant. **Causes:** context size grew (k raised, longer chunks, chat history unbounded); retries loop; one tenant / bot abusing; agent loops; more expensive model in use. **Fix:** token budget per request, summarise history, model routing (cheap model for easy queries), semantic cache, prompt caching, per-user rate limits & quotas, max-iterations on agents. **Prevent:** cost per request dashboard + budget alerts.

12. Different answers in English vs Hindi (or other language) for the same question

Causes: embedding model weak at cross-lingual retrieval; docs only in English; LLM weaker in that language. **Fix:** multilingual embedding model; translate query → retrieve in doc language → answer in user's language; eval per language.

B. LLM apps & agents

13. The model suddenly got worse, and we didn't change anything

Causes: provider updated the model behind an alias (`gpt-x-latest`); our data/docs changed; user traffic mix changed (new customer, new feature); a dependency (tokenizer, SDK default params) changed; system prompt edited via config by another team. **Isolate:** rerun golden eval set now vs. stored results; check model version in response headers/logs; diff prompt/config history. **Fix:** pin dated model versions; version prompts in git; rollback. **Prevent:** daily scheduled eval run against production config; alert on score drop.

14. The model sometimes returns invalid JSON and breaks the downstream service

Fix (layered): use structured output / JSON mode / function calling with schema; validate with a schema (Pydantic); on failure → retry with the error message ("fix this JSON"); fallback default; temperature 0; keep schema small. **Prevent:** parse-failure rate metric; contract tests.

15. Prompt injection: a user made the bot reveal its system prompt / ignore rules

Explain: direct injection (user input) vs **indirect injection** (malicious text inside retrieved doc/web page/email). **Mitigate:** never trust LLM output for authorization; least-privilege tools (read-only by default, human approval for actions like send/delete/pay); separate system vs. untrusted content clearly; input/output classifiers; don't put secrets in prompts; allow-listed tool arguments. **Key line:** "*I assume the model WILL be tricked, so I design the system so a tricked model can't do damage.*"

16. Agent gets stuck in a loop calling the same tool

Causes: tool returns error the model doesn't understand; ambiguous tool descriptions; no stopping criterion. **Fix:** max-steps + max-cost limits, detect repeated identical calls, clear error messages from tools, better tool descriptions, planner/executor split, return partial result + escalate to human.

17. LLM provider is down / rate limiting us (429s)

Mitigate: retries with exponential backoff + jitter; **fallback to a second provider/model** via an LLM gateway; queue non-urgent requests; graceful degradation ("Answers delayed"). **Long-term:** multi-provider gateway, quota management per feature, circuit breaker, self-hosted backup model for critical paths.

18. Chatbot forgets earlier parts of a long conversation

Cause: history truncated to fit context window. **Fix:** rolling summary of older turns + last N turns verbatim; store facts in memory store (vector or key-value) and retrieve relevant ones; per-session state in Redis.

19. How do you know your LLM feature is actually good? (evaluation)

Answer layers: (1) offline golden set with references → exact-match/semantic similarity, LLM-as-judge with rubric (calibrated against human labels), (2) RAG-specific: context recall/precision, faithfulness, answer relevance, (3) online: thumbs up/down, resolution rate, escalation-to-human rate, A/B tests, (4) safety: red-team set, toxicity, PII leakage. **Gotcha:** LLM judges are biased (position, verbosity, self-preference) → randomize order, validate against humans.

20. Outputs contain PII (emails, phone numbers) of other customers

Mitigate: output PII filter now. **Root causes:** PII in indexed docs, or in fine-tuning data (model memorized it), or shared cache/memory across users. **Fix:** PII redaction at ingestion, tenant isolation, cache scoping, data retention policies, remove from training data and retrain.

21. Fine-tuned model is great on our eval but users say it's worse at general tasks

Cause: catastrophic forgetting / overfitting to narrow data; eval set too narrow; eval leak (test examples in train). **Fix:** mix general data in fine-tuning, LoRA with lower rank/lr, fewer epochs, evaluate on general benchmark too, deduplicate train/test. Ask: "Did we even need fine-tuning, or would RAG/prompting work?"

22. Responses are too long / verbose and users complain

Fix: explicit length instruction + examples, `max_tokens`, output format constraints, eval metric for length, post-processing summarizer as last resort.

C. ML models in production

23. Model accuracy was 95% offline but 70% in production

Causes (ranked): 1. **Train/serve skew** — features computed differently online vs offline (different code, units, defaults for nulls, timezones). 2. **Data leakage** in training — a feature that wouldn't be available at prediction time (e.g., "refund_issued" when predicting fraud). Offline score was fake. 3. **Distribution shift** — production population differs from training data. 4. Wrong split — random split on time-series data (future leaked into training). 5. Preprocessing mismatch — different tokenizer/scaler version, label encoding order. **Isolate:** log production features → recompute the same rows offline → compare feature-by-feature. Check feature importance for suspiciously strong features. **Fix:** single feature definition shared by training & serving (**feature store**), time-based splits, point-in-time correct joins. **Prevent:** skew monitoring, shadow deployment before launch.

24. Model performance slowly degrades over months

Cause: data drift (input distribution changes) or **concept drift** (relationship between input and label changes, e.g., fraud patterns evolve). **Detect:** monitor input feature distributions (PSI, KL divergence), prediction distribution, and actual performance when labels arrive (often delayed!). **Fix:** scheduled retraining or drift-triggered retraining, sliding window training data, online learning for fast-moving domains.

25. Model's predictions suddenly all the same (e.g., everything "not fraud")

Causes: upstream feature broken → all nulls → default value; a feature service down and returning fallback; model file corrupted/wrong version deployed; input scaling bug. **Isolate:** check prediction distribution & feature null rates around the time it started; correlate with deploys. **Fix:** rollback; fail loudly (alert) instead of silently defaulting. **Prevent:** input validation, alerts on prediction distribution, canary deploys.

26. Labels arrive 30 days late (e.g., loan default, chargebacks). How do you monitor?

Answer: use proxy signals until labels arrive — input drift, prediction drift, early indicators (first payment missed), human-reviewed sample. Then backfill real performance when labels land.

27. Fraud model: 99.9% accuracy but catches no fraud

Cause: class imbalance — predicting "not fraud" always is 99.9% accurate. **Fix:** use precision/recall, PR-AUC, recall at fixed false positive rate; class weights, resampling, threshold tuning based on business cost (cost of missed fraud vs. cost of blocking good user).

28. New model won the offline evaluation but lost the A/B test

Causes: offline metric ≠ business metric; offline data biased by old model (it only saw items the old model showed — **selection bias / feedback loop**); latency increase hurt engagement; novelty effects; A/B test underpowered. **Fix:** align offline metric to online goal, counterfactual/off-policy evaluation, check guardrail metrics (latency), run test longer/with power analysis.

29. Model is biased against a user group

Isolate: slice metrics by group (FPR, FNR per group). **Causes:** under-representation in training data, proxy features (zip code ≈ race), biased historical labels. **Fix:** rebalance/collect data, remove proxies, fairness constraints, per-group thresholds (with legal review), human review for high-stakes.

30. Retraining pipeline produced a worse model and it auto-deployed

Fix: never auto-deploy without gates: eval against current champion on fixed holdout + recent data, sanity checks (prediction distribution), canary with automatic rollback. Model registry with champion/challenger.

31. Can't reproduce last month's model

Need: version everything — code (git), data snapshot (DVC/Delta/lakehouse time travel), features, hyperparameters, random seeds, library versions (Docker image), experiment tracking (MLflow/W&B).

32. Model works in batch, but real-time version is different

Cause: batch features use full-day aggregates; real-time uses partial windows; different code paths. **Fix:** same feature definitions (feature store, streaming computation), point-in-time features, compare outputs on the same entities.

D. Data & pipelines

33. A daily pipeline silently produced half the usual rows

Fix/prevent: data quality checks (row counts vs expected range, null rates, schema, freshness) that **fail the pipeline** (Great Expectations / dbt tests), alerting, idempotent reruns.

34. Upstream team changed a column type and broke our model

Fix: data contracts (schema agreed and validated at boundary), schema registry, version columns instead of mutating, CI tests on upstream changes.

35. Duplicate events in Kafka caused double-counting

Explain: most queues give *at-least-once* delivery. **Fix:** idempotent consumers (dedupe by event_id with TTL store), idempotent writes (upsert by key), exactly-once semantics where supported.

36. Training data has labeling errors

Fix: inter-annotator agreement, label review on disagreements, confident learning (find examples the model strongly disagrees with), gold questions for labelers.

37. Backfill job overloaded the production database

Fix: read from replica/warehouse not primary, rate-limit batch jobs, run off-peak, separate OLTP and analytics.

38. Embedding / ingestion job takes 3 days for the full corpus

Fix: parallelize with workers on a queue, batch embedding calls, GPU batching, incremental updates only (hash content → skip unchanged), checkpoint & resume.

E. Serving, infra & scale

39. Model latency p99 spikes every few minutes

Causes: garbage collection pauses, autoscaling cold starts (model loading), noisy neighbour on GPU, batch jobs sharing the cluster, large inputs, cache expiry thundering herd. **Fix:** keep warm replicas / min instances, preload models, dynamic batching with max wait, separate pools for batch vs online, timeout + fallback.

40. Traffic goes 10× during a product launch

Plan: horizontal autoscaling on stateless servers, queue + backpressure, rate limiting & priority tiers, cache hot results, pre-warm capacity, graceful degradation (smaller model, cached answers), load test beforehand.

41. GPU cost is too high for LLM inference

Fix: continuous batching (vLLM/TGI), quantization (INT8/INT4), smaller distilled model, routing easy queries to small model, KV cache/prefix caching, spot instances for batch workloads, autoscale to zero for low traffic, right-size GPUs.

42. One replica returns different predictions than others

Cause: mid-rollout with mixed versions, stale model file on one node, different hardware/library version, local cache. **Fix:** immutable versioned model artifacts, readiness checks that verify model hash, version in response header.

43. Rolled out a new model, error rate jumped — what now?

Answer: rollback first (that's why we canary + keep previous version warm), then debug from logs offline. "Mitigate first, investigate second."

44. Vector DB is the bottleneck at 50M vectors

Fix: shard by tenant/hash, tune HNSW (M, ef), use IVF-PQ/quantization to cut memory, metadata pre-filter partitions, replicas for read throughput, tiered storage (hot/cold).

45. A downstream feature service is slow, making all predictions slow

Fix: timeouts + defaults for non-critical features, circuit breaker, cache features, precompute batch features, async calls in parallel.

46. Users in another region experience high latency

Fix: deploy in multiple regions, route by geo-DNS, replicate read data/indexes, edge cache, smaller models at edge. Trade-off: consistency of data across regions.

F. Recommenders, search & ranking

47. Recommendations show the same popular items to everyone

Cause: popularity bias + feedback loop; cold-start fallback used too often; candidate generation too narrow. **Fix:** diversity/novelty in re-ranking, exploration (bandits, epsilon-greedy), personalization features, debias training with exposure weighting.

48. New users / new items get bad recommendations (cold start)

Fix: content-based features (item text/image embeddings), onboarding questions, popularity within segment, contextual features (location, device), explore budget for new items.

49. Search results are relevant but click-through is dropping

Isolate: UI change? latency? new content mix? query distribution shift? **Fix:** check position bias in training data, freshness features, A/B test ranker, analyze zero-click queries.

50. Users found the recommender is gaming engagement (clickbait rising)

Cause: optimizing a proxy (clicks) not the true goal (satisfaction). **Fix:** multi-objective ranking (dwell time, return visits, hides/reports as negatives), quality classifiers, long-term holdout metrics.

Final drill — 10 rapid-fire questions (answer in 30 seconds each)

1. Same chunks retrieved, different answers → *generation randomness / prompt diff*

2. Different chunks retrieved for the same query → *ANN non-determinism, query rewrite, index version, filters*
3. Great offline, bad online → *train/serve skew, leakage, drift*
4. Slow degradation → *drift → monitor + retrain*
5. Sudden drop → *something changed → correlate with deploys/data*
6. All predictions identical → *broken feature upstream*
7. Provider down → *retry, fallback provider, circuit breaker*
8. Answer is outdated → *ingestion lag, old chunks not deleted, cache*
9. Data leak between users → *security incident, pre-filter ACLs, scoped cache*
10. Cost spike → *token breakdown, context growth, loops, abuse → budgets + routing + cache*

05 — Case Studies: Full Interview Answers

Each case follows the same 7-step structure you should use in the interview (see 06-interview-playbook.md):

> **1. Requirements → 2. Estimates → 3. API → 4. High-level design → 5. Data model → 6. Deep dives → 7. What-ifs & trade-offs**

AI/ML cases

1. Enterprise RAG assistant (multi-tenant) ⭐ most asked
2. Customer-support AI agent
3. LLM gateway / internal LLM platform
4. Recommendation system (video/product feed)
5. Real-time fraud detection
6. Semantic search for e-commerce
7. Content moderation system
8. Document AI pipeline (invoices/forms extraction)

Classic cases (also asked to AI engineers)

9. URL shortener
10. Chat application
11. Rate limiter

---

1. Enterprise RAG assistant (multi-tenant)

> "Design a ChatGPT-like assistant that answers questions over each company's internal documents."

1. Requirements - Functional: upload/connect docs (PDF, Confluence, Drive), ask questions in chat, answers with citations, follow-up questions, respect document permissions, admin sees usage. - Non-functional: answers grounded (low hallucination), p95 time-to-first-token < 2 s, strict tenant isolation, docs searchable within 15 min of update, 99.9% availability, cost control.

2. Estimates — 500 companies × 2,000 users × 10 questions/day = 10M/day ≈ 115 QPS avg, ~350 peak. 500 tenants × 200k docs × 20 chunks = 2B chunks → 2B × 3 KB (768-dim fp32) ≈ 6 TB → shard + quantize. Tokens: 10M × 4k ≈ 40B tokens/day → **cost is the #1 concern** → caching, routing.

3. API

```
POST /v1/sources                    connect a source / upload file → returns job_id
GET  /v1/jobs/{id}                  ingestion status
POST /v1/conversations              create chat
```

```
POST /v1/conversations/{id}/messages { "text": "..." } → SSE stream of tokens + citations
POST /v1/messages/{id}/feedback      { "rating": -1, "comment": "..." }
```

4. High-level design

```
                      ┌──────────────── INGESTION (async) ────────────────┐
Sources/connectors → │ Queue → Parser/OCR → Chunker → Embedder (GPU batch) │ → Vector DB
(sharded by tenant)
  (webhooks / CDC)   │                                  → Keyword index    │ → Object store
(raw docs)
                      └─────────────────────────────────────────────────────┘ → Metadata DB
(Postgres: docs, ACLs, versions)

User → API gateway (auth, tenant, rate limit) → Chat service
          Chat service → Conversation store (history)
                       → Query rewriter (small LLM)
                       → Retriever (hybrid, ACL pre-filter by tenant + groups)
                       → Reranker
                       → Context builder (token budget)
                       → LLM gateway (routing, fallback, caching) → LLM providers / self-hosted
vLLM
                       → Output guardrails + citation verification
                       → SSE stream to user
          All steps → Tracing/logging (versions, chunk IDs, tokens, cost) → Eval & monitoring
```

5. Data model - `documents(doc_id, tenant_id, source, title, version, updated_at, acl_groups[], status)` - `chunks(chunk_id, doc_id, tenant_id, text, section_path, embedding, acl_groups[], index_version)` - `conversations(conv_id, user_id, tenant_id)` / `messages(msg_id, conv_id, role, text, citations, model_version, prompt_version, index_version, latency, tokens)`

6. Deep dives - **Tenant isolation:** namespace per tenant in vector DB (or index per large tenant); every query has mandatory `tenant_id` filter enforced server-side; ACL groups as pre-filter; cache keys include tenant + permission scope. - **Freshness:** connectors push change events → re-ingest only changed docs (content hash) → delete old chunks by doc_id → insert new. Full re-embeds (model change) build a new index version and swap alias. - **Quality:** hybrid search + rerank; contextual chunk headers; "answer only from context with citations"; verify citations; say "I don't know" when retrieval confidence is low. - **Latency:** streaming, parallel retrieval, rerank only top-30, small model for query rewrite, prompt caching. - **Cost:** semantic cache for repeated questions per tenant, model routing (simple → small model), context budget, per-tenant quotas. - **Evaluation:** per-tenant golden sets from real queries; retrieval recall@k, faithfulness, consistency; thumbs-down sampling to human review; CI eval gate on prompt/model changes.

7. What-ifs → see scenarios 1–12 in 04-what-if-scenarios.md. Be ready for: inconsistent answers, no answer, data leakage between tenants, outdated answers, embedding model migration, provider outage.

---

2. Customer-support AI agent

> "Design an AI agent that resolves support tickets: answers questions, checks order status, issues refunds."

- **Requirements:** chat + email channels, answer FAQs (RAG over help center), take actions via tools (order lookup, refund ≤ $50), hand off to human, audit trail. Non-functional: safe actions, < 3 s responses, measurable resolution rate.
- **Design:**

```
Channel adapters → Conversation service → Intent classifier/router
   ├─ FAQ → RAG pipeline
   ├─ Action → Agent (LLM + tools: get_order, create_refund, update_address)
   │            Tools behind a Tool API with authZ (acts AS the customer, not admin), limits,
idempotency keys
   │            Risky actions (refund > $50) → human approval queue
   └─ Escalation → Human agent console (with AI summary of conversation)
State: conversation store + ticket system;  Observability: trace every tool call
```

- **Deep dives:** idempotent refunds (idempotency key per ticket), policy checks done in code not in prompt, max-steps limit, prompt-injection defense (user can't make it refund $5,000 because the tool enforces limits), evaluation via simulated customers + replay of historical tickets, metrics: resolution rate, CSAT, escalation rate, wrong-action rate.
- **What-ifs:** agent loops (#16), injection (#15), tool API down (fallback to human), duplicate refund (idempotency).

---

3. LLM gateway / internal LLM platform

> "Many teams in our company want to use LLMs. Design a platform."

- **Requirements:** one API for many models/providers, auth per team, quotas & cost attribution, logging/redaction, fallback, caching, eval & prompt management.
- **Design:**

```
Teams' apps → LLM Gateway (stateless, autoscaled)
                ├─ AuthN/Z (API keys per team), PII redaction
                ├─ Rate limiter (tokens/min per team, Redis token bucket)
                ├─ Router (model by policy/cost/latency; fallback on 429/5xx; circuit breaker)
                ├─ Cache (exact + semantic)
                ├─ Providers: external APIs, self-hosted vLLM cluster (GPU autoscaling)
                └─ Async logging → Kafka → warehouse (usage, cost per team) + trace store
Prompt registry (versioned prompts) | Eval service (run suites on new model/prompt versions)
```

- **Trade-offs:** central gateway adds a hop (~ms) and is a critical dependency → deploy multi-AZ, keep it thin. Streaming support required end to end.

---

4. Recommendation system

> "Design the home feed recommendations for a video app with 100M users."

- **Requirements:** personalized feed, fresh content, < 200 ms, optimize long-term engagement (watch time, not just clicks), diversity.
- **Estimates:** 100M DAU × 20 feed loads = 2B/day ≈ 23k QPS avg, ~70k peak. 10M candidate videos.
- **Design (multi-stage funnel):**

```
Request → Candidate generation (≈1000 items, parallel sources):
              ├─ Two-tower model: user embedding → ANN search over video embeddings
              ├─ Collaborative filtering / co-watch
              ├─ Trending / fresh / followed creators
          → Ranking model (≈1000 → 100): deep model with user, item, context features; multi-
task (P(click), P(watch>50%), P(like), P(hide))
          → Re-ranking (business rules): diversity, freshness, dedupe, remove watched, policy
filters
          → Feed
Features: online feature store (Redis) ← streaming (Kafka/Flink: recent watches) + batch (Spark
daily)
Training: logged impressions + engagement → daily/continuous training → registry → canary → A/B
test
```

- **Deep dives:** cold start (#48), popularity bias & exploration (#47), position bias in logged data, multi-objective scoring (`score = w1·P(watch) + w2·P(like) − w3·P(hide)`), precomputing for inactive users (batch) vs real-time for active.
- **What-ifs:** offline win/online loss (#28), clickbait (#50), feature pipeline lag → stale recommendations.

---

5. Real-time fraud detection

> "Score every card transaction for fraud in < 100 ms."

- **Requirements:** score at authorization time, block/allow/review decision, explainability for analysts, adapt to new fraud patterns. 10k TPS peak.
- **Design:**

```
Payment service → Fraud API (p99 < 100ms)
     ├─ Rules engine (hard blocks: blacklisted card, impossible velocity)
     ├─ Feature fetch: online store (Redis) — real-time aggregates (txns in last 5 min, new
device?, distance from last txn)
     │      ↑ Streaming pipeline (Kafka → Flink) updates aggregates per card/user/merchant
     ├─ Model (GBDT e.g. XGBoost — fast, strong on tabular; maybe + graph features for fraud
rings)
     └─ Decision: score → thresholds (allow / step-up auth (OTP) / block / manual review queue)
Labels: chargebacks (delayed 30–90 days), analyst reviews → training data (point-in-time
features)
```

- **Deep dives:** class imbalance (#27), delayed labels (#26), threshold set by business cost, **fail-open vs fail-closed** if fraud service is down (usually fail-open with rules-only fallback for small amounts — state the trade-off), adversarial drift (#24), explainability (SHAP for analysts).

---

6. Semantic search for e-commerce

> "Users type 'comfortable running shoes for flat feet' — return relevant products."

- **Design:** query understanding (spell-fix, category/attribute extraction with an LLM or classifier: category=running shoes, feature=arch support) → **hybrid retrieval** (BM25 on title/attributes + vector search on product embeddings) with filters (in stock, price, size) → learning-to-rank model (features: text relevance, popularity, CTR, conversion, price, personalization) → re-rank (diversity, sponsored slots).
- Indexing: product catalog changes → CDC → update search index (Elasticsearch/OpenSearch with vector support).
- Evaluation: nDCG on labeled query-product pairs, online CTR / add-to-cart / zero-result rate.
- What-ifs: zero results (#2 analog: query relaxation, fallbacks), position bias (#49), new products cold start (#48).

---

7. Content moderation system

> "Detect harmful posts (text + images) on a social platform with 50M posts/day."

- **Design:**

```
Post created → sync fast check (hash match against known-bad content, cheap text classifier) →
publish or hold
             → async deeper models via queue (multimodal classifiers / LLM with policy prompt)
             → score per policy category → auto-remove (high confidence) | human review queue
(uncertain) | allow
User reports → priority boost into review queue
Human decisions → labels → retraining; appeals flow
```

- **Deep dives:** precision vs recall per category (CSAM: maximize recall; satire: precision), cost (cheap model filters, expensive model only for uncertain), multilingual, adversarial evasion (leetspeak, text in images → OCR), reviewer wellbeing & throughput, latency of the async path.

---

8. Document AI pipeline

> "Extract fields from 1M invoices/month (vendor, date, total, line items)."

- **Design:** upload → object storage → queue → classify doc type → OCR/layout model (or multimodal LLM) → field extraction (structured output with JSON schema) → validation rules (totals sum, date

formats, vendor in master data) → confidence score → auto-approve or human review UI → corrections stored as training data → ERP via API.
- **Estimates:** 1M/month ≈ 0.4/s average — throughput easy; focus on accuracy, cost per document, and review workload.
- **What-ifs:** new invoice layout breaks extraction (drift → human review rises → retrain/few-shot examples), scanned low-quality docs (pre-processing, OCR confidence), duplicate invoices (hash + business key dedupe), LLM returns invalid JSON (#14).

---

9. URL shortener

- **Req:** shorten URL, redirect fast, 100M new URLs/month, read-heavy (100:1).
- **Estimates:** writes ≈ 40/s, reads ≈ 4k/s; 5 years × 1.2B × 500 B ≈ 3 TB.
- **Design:** `POST /shorten` → ID generator (Snowflake or counter ranges per server) → **base62 encode** (7 chars = 62^7 ≈ 3.5T) → store `code → long_url` in key-value DB (DynamoDB/Cassandra). `GET /{code}` → cache (Redis) → DB → **301/302 redirect**. Analytics via async events to Kafka.
- **Trade-offs:** hash vs counter (collisions vs coordination), 301 (cached by browser, less analytics) vs 302.

10. Chat application

- **Req:** 1:1 & group chat, online status, delivery receipts, history.
- **Design:** clients hold **WebSocket** connections to chat servers; connection registry (Redis: user → server); message → chat server → persist (Cassandra, partitioned by conversation_id, sorted by time) → route to recipient's server via pub/sub; offline users → push notification; message IDs time-sortable (Snowflake) for ordering.
- **Deep dives:** at-least-once delivery + client dedupe, group fan-out, presence via heartbeats, end-to-end encryption.

11. Rate limiter

- **Design:** token bucket per (user, endpoint) stored in Redis with atomic Lua script; at API gateway; return 429 + `Retry-After`. For LLM: limit tokens/min and concurrent requests.
- **Trade-offs:** centralized Redis (accurate, extra hop) vs local in-memory (fast, approximate); fail-open if Redis down.

06 — Interview Playbook & Cheat Sheet

A. The 45-minute "Design X" script

Time	Step	What you say / do
0–5 min	**Requirements**	"Let me clarify scope." Functional (3–5 features), non-functional (scale, latency, accuracy, cost, privacy). Write them down. Ask: *who are the users, how many, what matters most?*
5–8 min	**Estimates**	QPS, storage, tokens/cost. Then **say what it means**: "So cost, not QPS, is the bottleneck."
8–12 min	**API**	3–5 endpoints. Mention streaming for LLMs.
12–25 min	**High-level design**	Draw boxes & arrows. Walk one request end to end. For ML: show **offline** (data, training/ingestion) and **online** (serving) paths separately.
25–38 min	**Deep dives**	Pick 2–3 hardest parts (or ask: "Which part would you like me to go deeper on?"). Data model, retrieval/ranking, scaling, evaluation.
38–45 min	**What-ifs, trade-offs, monitoring**	Failure modes, bottlenecks, how you'd evaluate & monitor, what you'd do with more time.

B. The "What if X happens?" script

Use **C.L.H.I.F.P.** from 03-debugging-framework.md: 1. **Clarify** the symptom (who / when / since when / what changed). 2. **Layer map** — draw the pipeline. 3. **Hypotheses** ranked by likelihood. 4. **Isolate** — cheapest test first; find the first layer where output differs. 5. **Fix** — mitigation now + root-cause fix later. 6. **Prevent** — evals, monitoring, alerts, versioning.

C. Things interviewers secretly score

- ✅ You asked clarifying questions before designing.
- ✅ You stated assumptions out loud.
- ✅ You drove the conversation (didn't wait to be led).
- ✅ You explained **trade-offs** ("I chose X over Y because…, the cost is…").
- ✅ You covered evaluation and monitoring (huge for ML roles).
- ✅ You started simple, then scaled ("for v1 I'd use pgvector; at 100M vectors I'd move to…").
- ✅ You handled pushback calmly ("Good point — in that case I'd change…").
- ❌ Jumping into a model architecture before understanding the problem.
- ❌ Buzzword lists without reasons.
- ❌ Silence. ❌ Over-engineering for 5 QPS. ❌ Ignoring cost/latency of LLMs.

D. Trade-off sentences (memorize the shape)

- "**X gives us** lower latency, **at the cost of** staleness, **which is acceptable here because** …"

- Latency vs quality: bigger model / reranker / more context = better but slower and costlier.
- Freshness vs cost: real-time ingestion vs hourly batch.
- Consistency vs availability: payments → consistency; feeds → availability.
- Precision vs recall: depends on the cost of each error type.
- Build vs buy: managed vector DB/LLM API (speed) vs self-host (control, cost at scale).
- RAG vs fine-tune: knowledge vs behavior.
- Determinism vs creativity: temperature 0 for factual Q&A, higher for brainstorming.

E. One-page cheat sheet

Building blocks: LB · API gateway · cache (Redis) · CDN · SQL · NoSQL · object storage · queue (SQS/Rabbit) · stream (Kafka) · search (Elastic) · vector DB · feature store · model registry · model server · LLM gateway · monitoring/tracing.

Scale moves: stateless + horizontal · cache · read replicas · shard · async queue · CDN · batch on GPU · autoscale · multi-region.

Reliability moves: redundancy · timeouts · retries with backoff + jitter · circuit breaker · fallback/degradation · idempotency · canary + rollback · rate limits · DLQ.

ML moves: feature store (no skew) · point-in-time joins · time-based splits · shadow → canary → A/B · drift monitoring · champion/challenger · version code+data+model.

LLM/RAG moves: hybrid search · rerank · good chunking + contextual headers · query rewrite · ACL pre-filter · citations · "I don't know" threshold · temp 0 + pinned model for consistency · versioned index + alias swap · semantic/prompt cache · model routing · streaming · token budgets · guardrails · evals in CI (recall@k, faithfulness, consistency).

Numbers: 1M/day ≈ 12 QPS · 1 day ≈ 10^5 s · 1 token ≈ 4 chars · 1536-d fp32 vector ≈ 6 KB · 7B fp16 ≈ 14 GB · Redis ~1 ms · LLM TTFT ~0.5 s · 99.9% = 8.8 h/yr down.

F. Common questions to prepare answers for

1. Design a RAG system for internal documents. *(05 #1)*
2. Design a chatbot/agent for customer support. *(05 #2)*
3. How would you evaluate an LLM application? *(02 §10)*
4. How do you reduce hallucinations? *(04 #3)*
5. RAG vs fine-tuning — when each? *(02 §12)*
6. How do you reduce LLM latency / cost? *(04 #6, #11, 02 §8)*
7. Design a recommendation system. *(05 #4)*
8. Design fraud detection. *(05 #5)*
9. Model works offline but not in production — why? *(04 #23)*
10. How do you detect and handle drift? *(04 #24)*
11. How do you deploy a new model safely? *(02 §5, 04 #43)*
12. Same question, different answers — why? *(03 worked example)*
13. How to handle multi-tenant data isolation? *(04 #7)*
14. How to serve a 70B model? *(02 §8, 04 #41)*

15. How do you defend against prompt injection? *(04 #15)*

G. Your 2-week mock interview plan

Day	Task
1, 3, 5, 7, 9, 11, 13	One full 45-min design (rotate 05 cases), **recorded**. Watch back: Did I clarify? Estimate? Trade-offs? Monitoring?
2, 4, 6, 8, 10, 12, 14	8 scenario questions from 04, 3 min each, out loud, timed
Daily	10 min: redraw the RAG layer map and ML lifecycle from memory

Practice partners: a friend, a peer engineer, or ask an AI to play interviewer: "*Act as a senior ML interviewer. Ask me a system design question, then push back with 'what if' failure scenarios. Score me on structure, trade-offs, and depth.*"

H. If you blank out in the interview

Say: "*Let me take a step back and walk through the request path.*" — then draw the pipeline. Your layer map will give you the next thing to say. Every time.

07 — Machine Learning Fundamentals (Theory Round)

Format: **Q → short answer you can say in 30–60 seconds.** Then "what if" scenarios at the end. Practice: cover the answer, say it out loud, compare.

Contents

1. Core concepts
2. Bias, variance, regularization
3. Linear & logistic regression
4. Trees & ensembles
5. Other classic algorithms
6. Unsupervised learning
7. Evaluation & metrics
8. Data preparation & feature engineering
9. What-if scenarios (ML)

---

1. Core concepts

Q: Supervised vs unsupervised vs self-supervised vs reinforcement learning? Supervised: learn from labeled (x, y). Unsupervised: find structure without labels (clustering, dimensionality reduction). Self-supervised: create labels from the data itself (predict next token, masked word) — how LLMs are pretrained. RL: an agent learns by trial and error from rewards.

Q: Classification vs regression? Predict a category vs a continuous number.

Q: Parametric vs non-parametric? Parametric has a fixed number of parameters (linear regression). Non-parametric grows with data (kNN, decision trees can).

Q: Generative vs discriminative? Discriminative models P(y|x) (logistic regression). Generative models P(x, y) or P(x) (Naive Bayes, GPT, diffusion).

Q: What is a loss function vs cost vs objective? Loss: error for one example. Cost: average over dataset. Objective: what we optimize (cost + regularization).

Q: What is gradient descent? Iteratively move parameters opposite to the gradient of the loss: `w = w − lr · ∇L`. Batch (all data), stochastic (one sample), mini-batch (common).

Q: Why train / validation / test split? Train to fit, validation to tune hyperparameters/early stop, test to estimate final performance once. Using test for tuning = leakage → optimistic results.

Q: What is cross-validation? Split data into k folds; train on k−1, validate on 1, rotate; average. Gives a more reliable estimate with small data. Use **stratified** k-fold for imbalance, **time-series split** for temporal data, **group k-fold** when same user appears multiple times.

Q: Curse of dimensionality? In high dimensions data becomes sparse and distances become similar, so distance-based methods (kNN) degrade and you need far more data.

Q: No Free Lunch theorem? No single algorithm is best for all problems — choose by data and constraints.

2. Bias, variance, regularization

Q: Bias-variance trade-off? Bias = error from too-simple assumptions (underfitting). Variance = error from sensitivity to training data (overfitting). Total error = bias^2 + variance + noise. More complexity → lower bias, higher variance.

Q: How do you detect overfitting / underfitting? | | Train error | Validation error | |---|---|---| | Underfitting | high | high | | Overfitting | low | high (big gap) | | Good fit | low | low, small gap |

Q: How to fix overfitting? More data, data augmentation, simpler model, regularization (L1/L2, dropout), early stopping, feature selection, ensembling/bagging, cross-validation for tuning.

Q: How to fix underfitting? More complex model, more/better features, less regularization, train longer.

Q: L1 vs L2 regularization? L1 (Lasso) adds |w| → pushes weights to exactly zero → feature selection. L2 (Ridge) adds w^2 → shrinks weights smoothly, handles correlated features. Elastic Net = both.

Q: Why does L1 create sparsity? Its penalty has a constant gradient (corner at zero in the constraint diamond), so the optimum often lands on an axis where weights are exactly zero.

3. Linear & logistic regression

Q: Assumptions of linear regression? Linearity, independent errors, homoscedasticity (constant error variance), normally distributed errors (for inference), no strong multicollinearity.

Q: What is multicollinearity and why does it matter? Features highly correlated → unstable, uninterpretable coefficients. Detect with VIF; fix with removal, PCA, or Ridge.

Q: Logistic regression — how does it work? Linear score → sigmoid → probability. Trained by minimizing **log loss (binary cross-entropy)**, which is maximum likelihood. Decision boundary is linear.

Q: Why not MSE for logistic regression? With sigmoid, MSE is non-convex and gives weak gradients; log loss is convex and penalizes confident wrong predictions strongly.

Q: R^2 meaning? Fraction of target variance explained by the model. Can be negative on test data if the model is worse than predicting the mean.

4. Trees & ensembles

Q: How does a decision tree split? Choose the feature/threshold that maximizes impurity reduction — Gini or entropy (information gain) for classification, variance/MSE for regression.

Q: Bagging vs boosting? - **Bagging** (Random Forest): train many models in parallel on bootstrap samples, average them → **reduces variance**. - **Boosting** (AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost): train models sequentially, each fixes previous errors → **reduces bias**. More prone to overfitting; needs tuning (learning rate, depth, n_estimators, early stopping).

Q: Why does Random Forest use random feature subsets? To decorrelate trees, so averaging reduces variance more.

Q: Why is XGBoost/LightGBM so popular for tabular data? Strong accuracy, handles missing values, regularization, fast (histogram-based splits), little preprocessing (no scaling needed). Usually beats deep learning on tabular data.

Q: Feature importance in trees — caveats? Impurity-based importance is biased toward high-cardinality features. Use permutation importance or SHAP.

Q: Stacking? Train a meta-model on predictions of several base models (use out-of-fold predictions to avoid leakage).

5. Other classic algorithms

Q: kNN? Predict using the k closest training points. Needs feature scaling; slow at prediction; suffers in high dimensions. Small k → overfit, large k → underfit.

Q: Naive Bayes? Bayes' rule assuming features are conditionally independent given the class. Fast, good baseline for text (spam). Use Laplace smoothing for unseen words.

Q: SVM? Find the hyperplane with maximum margin. Soft margin (C) allows errors. **Kernel trick** computes similarity in a higher-dimensional space without explicitly mapping (RBF, polynomial).

Q: When would you choose a simple model over a complex one? Small data, need interpretability (finance/health regulations), low latency, strong baseline, easier maintenance.

6. Unsupervised learning

Q: K-means? Pick k centroids, assign points to nearest, recompute centroids, repeat. Assumes spherical, similar-sized clusters; sensitive to init (use k-means++) and scaling. Choose k by elbow method / silhouette score.

Q: K-means vs DBSCAN vs hierarchical? DBSCAN finds arbitrary shapes and noise points, no k needed, but sensitive to eps. Hierarchical builds a dendrogram, good for small data.

Q: PCA? Projects data onto directions of maximum variance (eigenvectors of covariance matrix). Used for compression, visualization, removing correlation. Requires scaling. Linear only.

Q: t-SNE / UMAP? Non-linear dimensionality reduction for visualization. Distances between clusters in t-SNE are not meaningful.

Q: Anomaly detection methods? Statistical (z-score), Isolation Forest, One-class SVM, autoencoders (high reconstruction error = anomaly), density-based.

7. Evaluation & metrics

Q: Confusion matrix terms? TP, FP, TN, FN. - **Precision** = TP / (TP+FP) — "of what I flagged, how much was right?" (care when false alarms are costly: spam filter). - **Recall** = TP / (TP+FN) — "of all positives, how many did I catch?" (care when misses are costly: cancer, fraud). - **F1** = harmonic mean of precision and recall. - **Accuracy** misleading on imbalanced data.

Q: ROC-AUC vs PR-AUC? ROC plots TPR vs FPR across thresholds; AUC = probability a random positive ranks above a random negative. On heavy imbalance, ROC looks overly good — **PR-AUC** is more informative.

Q: How to choose a threshold? Based on business cost: minimize cost = FP·cost_FP + FN·cost_FN, or fix one constraint ("recall ≥ 95%") and maximize the other.

Q: Calibration? Predicted probability 0.8 should be right 80% of the time. Check with reliability diagram; fix with Platt scaling or isotonic regression. Matters when probabilities are used for decisions/pricing.

Q: Regression metrics? MAE (robust to outliers, interpretable), RMSE (penalizes big errors), MAPE (percent, breaks near zero), R^2.

Q: Ranking metrics? Precision@k, Recall@k, MRR, MAP, nDCG (accounts for position and graded relevance).

Q: Offline metric vs online metric? Offline (AUC on test set) is a proxy. Online (CTR, revenue, retention via A/B test) is what matters. They can disagree.

8. Data preparation & feature engineering

Q: How do you handle missing values? First ask *why* missing (random or meaningful?). Options: drop, impute (mean/median/mode, model-based, KNN), add "is_missing" indicator, use models that handle NaN (XGBoost).

Q: How to handle categorical features? One-hot (low cardinality), ordinal (ordered), target encoding (high cardinality — use out-of-fold to avoid leakage), hashing, learned embeddings.

Q: When do you need feature scaling? Distance/gradient-based models: kNN, SVM, k-means, PCA, linear/logistic regression with regularization, neural nets. Not needed for trees. Fit the scaler on **train only**.

Q: How to handle imbalanced data? Use proper metrics (PR-AUC, recall@precision); class weights; resampling (undersample majority, oversample minority, SMOTE — only on training data); threshold tuning; anomaly detection framing; collect more minority data.

Q: What is data leakage? Examples? Information in training that wouldn't be available at prediction time. Examples: scaling on full dataset before split; target encoding without folds; future data in time series; duplicate rows across train/test; a feature created after the outcome (e.g., "account_closed" predicting churn).

Q: Outliers? Investigate first (error or real?). Cap/winsorize, transform (log), robust models/losses (Huber, MAE), or remove if errors.

Q: Feature selection methods? Filter (correlation, mutual info), wrapper (recursive feature elimination), embedded (L1, tree importance), plus domain knowledge.

9. What-if scenarios

Your model has 98% training accuracy but 70% validation accuracy. What do you do?

Overfitting. Check leakage/split first (duplicates? distribution difference between train and val?). Then: regularization, simpler model, more data/augmentation, early stopping, cross-validation, feature selection. Plot learning curves: if val improves with more data → collect data.

Both training and validation accuracy are low.

Underfitting or bad data. Check labels quality, features informativeness, bugs (labels shuffled? wrong target column?). Try a more complex model, better features, less regularization. Compare to a simple baseline (predict majority class) — if you can't beat it, the problem is data, not model.

Validation score is suspiciously high (99%).

Suspect **leakage** first: target-derived features, duplicates across splits, future info, ID columns correlated with target. Check top feature importances — a single dominating feature is a red flag.

Model performs well on validation but poorly on test.

Overfit to validation through heavy tuning, or val/test come from different distributions. Use cross-validation, a fresh holdout, check distributions.

You have only 500 labeled examples.

Simple models + strong regularization, cross-validation, transfer learning / pretrained embeddings, data augmentation, active learning (label the most uncertain), weak supervision, semi-supervised learning, LLM-assisted labeling with human verification.

You have 1% positive class (fraud).

See §8 imbalance. Mention: never report accuracy; tune threshold on business cost; stratified splits.

Features are highly correlated.

Fine for prediction with trees; problematic for interpreting linear coefficients. Use regularization (Ridge), PCA, or drop redundant ones.

Model is accurate but stakeholders don't trust it.

Explainability: SHAP/LIME, feature importance, partial dependence; show examples; compare to simple interpretable baseline; calibrate probabilities; involve domain experts in error analysis.

Model works for most users but fails for one segment.

Slice-based error analysis; check data representation; add segment features or separate model; fairness considerations.

Training takes too long.

Sample data for iteration, fewer features, use LightGBM (fast), early stopping, distributed training, better hardware, cache preprocessing.

Labels are noisy.

Estimate noise (relabel a sample), consensus labeling, robust losses, remove examples model confidently disagrees with (confident learning), more regularization.

Data distribution changed after deployment.

Drift → see 04 #24.

08 — Deep Learning (Theory + Training Debugging)

Contents

1. Neural network basics
2. Optimization
3. Regularization & normalization
4. Architectures: CNN, RNN, Transformer
5. Training at scale
6. Generative models
7. What-if: training debugging scenarios ⭐ very commonly asked

---

1. Neural network basics

Q: What is a neuron / perceptron? Weighted sum of inputs + bias, passed through a non-linear activation.

Q: Why do we need non-linear activations? Without them, stacked linear layers collapse into a single linear function — the network couldn't learn non-linear patterns.

Q: Common activations? | Activation | Notes | |---|---| | Sigmoid | 0–1, saturates → vanishing gradients; used for binary outputs | | Tanh | −1–1, zero-centered, still saturates | | ReLU | max(0,x); fast, no saturation for x>0; can "die" (always 0) | | Leaky ReLU / PReLU | small slope for x<0 → fixes dying ReLU | | GELU / SiLU (Swish) | smooth; used in Transformers/LLMs | | Softmax | turns logits into a probability distribution (multi-class output) |

Q: Explain backpropagation. Forward pass computes outputs and loss. Backward pass applies the chain rule layer by layer from output to input to compute the gradient of the loss w.r.t. every weight. The optimizer then updates the weights.

Q: Vanishing / exploding gradients? Gradients multiplied through many layers shrink to ~0 (early layers stop learning) or blow up (NaN). Fixes: ReLU-family activations, good initialization (He/Xavier), normalization (BatchNorm/LayerNorm), residual connections, gradient clipping, LSTM/GRU gates for RNNs.

Q: Weight initialization — why not all zeros? All neurons would compute the same thing and get the same gradients (symmetry), so they never differentiate. Use Xavier/Glorot (tanh/sigmoid) or He/Kaiming (ReLU).

Q: Loss functions? MSE/MAE (regression), binary cross-entropy (binary), categorical cross-entropy (multi-class), focal loss (imbalance, detection), contrastive / triplet / InfoNCE (embeddings), KL divergence (distillation, VAEs).

Q: Why cross-entropy with softmax? Gradient becomes simply (predicted − true), stable and strong; corresponds to maximum likelihood.

2. Optimization

Q: SGD vs Momentum vs Adam? - SGD: noisy updates, may generalize well, needs tuned LR. - Momentum: accumulates velocity → faster, smoother through ravines. - RMSProp: per-parameter LR scaled by recent gradient magnitude. - **Adam**: momentum + RMSProp + bias correction; default choice. **AdamW**: decoupled weight decay — standard for Transformers.

Q: Learning rate — the most important hyperparameter. How to pick? LR range test (increase LR, watch loss), then use **warmup + decay** schedule (cosine, step, linear decay). Too high → diverge/NaN; too low → slow/stuck.

Q: Why warmup? Early in training Adam's statistics are unreliable and weights random; large updates can destabilize (especially Transformers). Warmup ramps LR up gradually.

Q: Batch size effects? Larger batch: faster per epoch on GPUs, smoother gradients, may generalize slightly worse, needs larger LR (linear scaling rule). Limited by memory → gradient accumulation simulates large batches.

Q: Local minima in deep learning? In high dimensions most critical points are saddle points; SGD noise and momentum help escape. Many minima are similarly good.

3. Regularization & normalization

Q: Dropout? Randomly zero activations during training (rate p) → prevents co-adaptation, acts like ensemble. Off at inference (activations scaled accordingly).

Q: BatchNorm vs LayerNorm? BatchNorm normalizes each feature across the batch — great for CNNs, depends on batch size, different behavior train vs eval (running stats). LayerNorm normalizes across features of each sample — independent of batch, used in Transformers/RNNs. RMSNorm: simpler LayerNorm used in modern LLMs.

Q: Other regularizers? Weight decay, data augmentation, early stopping, label smoothing, mixup/cutmix, stochastic depth.

Q: Residual (skip) connections — why? `output = F(x) + x` gives gradients a direct path backward → enables very deep networks (ResNet, Transformers) and makes it easy to learn identity.

4. Architectures

CNNs

Q: Why convolutions for images? Local connectivity + weight sharing → far fewer parameters, translation equivariance, hierarchical features (edges → textures → objects). **Q: Output size formula?** `(W - K + 2P) / S + 1`. **Q: Pooling?** Downsamples, adds small translation invariance. Max vs average. Global average pooling before classifier. **Q: 1×1 convolution?** Changes channel count cheaply, mixes channels

(used in Inception/ResNet bottlenecks). **Q: Receptive field?** Region of the input that affects one output unit; grows with depth, dilation, stride. **Q: Famous models?** LeNet → AlexNet → VGG (deep 3×3) → GoogLeNet/Inception → ResNet (skip connections) → EfficientNet (compound scaling) → ViT (Transformers on patches) → ConvNeXt.

RNNs

Q: RNN problem? Sequential (no parallelism), vanishing gradients over long sequences. **Q: LSTM vs GRU?** LSTM: cell state + input/forget/output gates → long-term memory. GRU: fewer gates (update/reset), faster, often similar performance.

Transformers

Q: Explain self-attention. Each token creates a Query, Key, Value. Attention weights = softmax($Q \cdot K^T / \sqrt{d}$) — how much each token should look at every other token. Output = weighted sum of Values. Captures long-range dependencies in parallel. **Q: Why divide by √d?** Dot products grow with dimension → softmax saturates → tiny gradients. Scaling keeps variance stable. **Q: Multi-head attention?** Several attention heads in parallel, each learns different relationships (syntax, coreference…), concatenated. **Q: Positional encoding?** Attention is order-agnostic, so we add position info: sinusoidal, learned, or **RoPE** (rotary, used in modern LLMs), ALiBi. **Q: Encoder vs decoder vs encoder-decoder?** Encoder-only (BERT): bidirectional, understanding tasks. Decoder-only (GPT, Llama): causal mask, generation. Encoder-decoder (T5, original Transformer): translation/summarization. **Q: Complexity of attention?** $O(n^2)$ in sequence length (time + memory). Solutions: FlashAttention (IO-aware exact), sparse/sliding-window attention, linear attention, KV caching for inference. **Q: Transformer block?** Multi-head attention → add & norm → feed-forward (MLP) → add & norm. Modern: pre-norm, RMSNorm, SwiGLU, RoPE, GQA.

5. Training at scale

- **Mixed precision** (fp16/bf16): faster, less memory; bf16 avoids overflow (no loss scaling needed); fp16 needs loss scaling.
- **Gradient accumulation:** simulate large batch.
- **Gradient checkpointing:** recompute activations in backward → less memory, ~30% slower.
- **Data parallel (DDP), FSDP/ZeRO, tensor parallel, pipeline parallel** — see 02 §3.
- **Transfer learning / fine-tuning:** freeze early layers, train head, then unfreeze with low LR. **PEFT/LoRA**: train small low-rank adapters instead of full weights.
- **Knowledge distillation:** small student learns from teacher's soft probabilities (with temperature).
- **Quantization / pruning** for inference.

6. Generative models

Q: Autoencoder vs VAE? Autoencoder compresses and reconstructs. VAE learns a probabilistic latent space (mean + variance) with KL regularization → can sample new data; uses reparameterization trick. **Q: GAN?** Generator vs discriminator in a min-max game. Problems: mode collapse, unstable training. **Q: Diffusion models?** Learn to reverse a gradual noising process — start from noise, denoise step by step. Stable, high quality (Stable Diffusion = diffusion in latent space + text conditioning via cross-attention,

classifier-free guidance). Slower sampling (fixes: fewer steps, distillation). **Q: Autoregressive models?** Generate one token at a time conditioned on previous (GPT).

---

7. What-if: training debugging scenarios

General debugging recipe (say this first):

> "Start small: overfit a single batch. If the model can't drive loss to ~0 on 1 batch, there's a bug, not a modelling problem. Then check data → model → loss → optimizer → evaluation, one at a time."

Loss is NaN / Inf

Causes: LR too high; exploding gradients; log(0) or division by zero in custom loss; fp16 overflow; bad input data (NaN in features, corrupted images); sqrt/exp of large values. **Fix:** lower LR, gradient clipping, add epsilon to log/division, use bf16 or loss scaling, check data for NaN (`torch.isnan(x).any()`), use `torch.autograd.detect_anomaly()`, use numerically stable built-ins (`CrossEntropyLoss` on logits, not softmax + log).

Loss doesn't decrease at all

Causes: LR too low/high; forgot `optimizer.zero_grad()` or `optimizer.step()`; frozen layers by mistake; labels misaligned with inputs (shuffled separately); wrong loss for task (softmax applied twice); dead ReLUs; data not normalized; bug in data pipeline (all same image). **Fix:** overfit one batch, visualize inputs + labels, check gradients are non-zero per layer, LR range test.

Loss decreases, then suddenly spikes

Causes: LR too high after warmup, bad batch (outlier/corrupt data), exploding gradients, numerical instability in fp16. **Fix:** gradient clipping, lower peak LR, longer warmup, skip/inspect the bad batch, bf16, restart from checkpoint.

Training loss goes down, validation loss goes up

Overfitting. Early stopping, more data/augmentation, dropout, weight decay, smaller model, mixup. Also check val pipeline matches train preprocessing.

Validation is better than training

Usually fine: dropout/augmentation are active only during training, train loss averaged over the epoch while val measured at the end. Also check val set isn't too easy/small or leaked.

Model predicts the same class for every input

Class imbalance (use weights/focal loss), LR too high causing collapse, dead network, labels broken, final activation wrong.

GPU out of memory (OOM)

Reduce batch size + gradient accumulation; mixed precision; gradient checkpointing; shorter sequences; smaller model / LoRA; free unused tensors (`del`, `torch.no_grad()` in eval — forgetting this stores activations!); avoid accumulating loss tensors with graph (`loss.item()`); FSDP/ZeRO offload.

Training is very slow / GPU utilization low (30%)

Data loading bottleneck → more `num_workers`, `pin_memory`, prefetching, preprocess offline, faster storage formats; too-small batch; CPU-GPU sync calls (`.item()` every step); use mixed precision, `torch.compile`, larger batch.

Results differ every run

Set seeds (python, numpy, torch, CUDA), deterministic algorithms (`torch.use_deterministic_algorithms`), fixed data order; some GPU ops are inherently non-deterministic. Report mean ± std over multiple seeds.

Model works in notebook but bad after deployment

`model.eval()` not called (dropout/BatchNorm in training mode!); different preprocessing (normalization constants, resize, tokenizer version); different input format (RGB vs BGR, channel order); quantization changed accuracy.

Fine-tuning a pretrained model makes it worse

LR too high (destroys pretrained weights → use 10–100× smaller LR), catastrophic forgetting, too few examples (freeze more layers, use LoRA), mismatched preprocessing vs pretraining.

Accuracy plateaus at a certain level

Check label noise ceiling (human agreement), error analysis on mistakes, more/better data often beats architecture changes, LR schedule (decay), bigger model if underfitting.

BatchNorm issues with batch size 1–2

Statistics unreliable → use GroupNorm/LayerNorm, freeze BN stats, or larger batch via accumulation (doesn't fix BN) / sync BN across GPUs.

09 — NLP & LLMs (Theory + Scenarios)

Contents

1. Classic NLP
2. Tokenization
3. Embeddings
4. Transformers & pretrained models
5. How LLMs are trained
6. Using LLMs: prompting, decoding, fine-tuning
7. LLM limitations & evaluation
8. What-if scenarios (NLP/LLM)

(For RAG and LLM system design, see 02 and 04.)

---

1. Classic NLP

Q: Text preprocessing steps? Lowercasing, tokenization, removing punctuation/stopwords, stemming (crude chop: "running"→"run") vs lemmatization (dictionary form: "better"→"good"), normalization. *Note:* for Transformers you mostly skip this — use the model's own tokenizer on raw text.

Q: Bag of Words vs TF-IDF? BoW counts words. TF-IDF = term frequency × inverse document frequency → down-weights common words. Both ignore word order and meaning.

Q: BM25? Improved TF-IDF ranking function (term frequency saturation + document length normalization). Still the strong baseline for keyword search.

Q: n-grams? Sequences of n words — capture some local order ("not good").

Q: Classic NLP tasks? Text classification, sentiment, NER (named entity recognition), POS tagging, parsing, machine translation, summarization (extractive vs abstractive), question answering, topic modeling (LDA), coreference resolution.

Q: What is NER and how to build it today? Labeling spans like PERSON/ORG/DATE. Token classification with BIO tags using a fine-tuned BERT-style model; or LLM with structured output for low-data cases.

Q: Language model perplexity? exp(average negative log-likelihood per token). Lower = model less "surprised". Comparable only with the same tokenizer.

2. Tokenization

Q: Word vs character vs subword tokenization? Word: huge vocab, out-of-vocabulary problem. Character: long sequences. **Subword** (BPE, WordPiece, Unigram/SentencePiece): frequent words whole, rare words split into pieces → no OOV, compact.

Q: How does BPE work? Start with characters, repeatedly merge the most frequent adjacent pair into a new token until vocab size reached.

Q: Why do tokenizers matter in interviews / production? - Cost and context limits are in tokens. - Non-English languages often use more tokens per word → higher cost, worse performance. - Numbers and code tokenized oddly → arithmetic/spelling weaknesses. - **Mismatched tokenizer between training and serving = garbage outputs.**

3. Embeddings

Q: Word2Vec? Learns word vectors by predicting context words (Skip-gram) or the word from context (CBOW). Similar words → close vectors; analogies (king − man + woman ≈ queen). GloVe: from co-occurrence matrix. FastText: subword n-grams → handles rare words.

Q: Static vs contextual embeddings? Static: one vector per word ("bank" same everywhere). Contextual (BERT, etc.): vector depends on the sentence.

Q: Sentence embeddings? Models trained with **contrastive learning** (pull similar pairs together, push others apart) — Sentence-BERT, E5, BGE, OpenAI/Cohere embeddings. Used for semantic search, clustering, RAG.

Q: Bi-encoder vs cross-encoder? Bi-encoder encodes query and document separately → precompute document vectors → fast retrieval. Cross-encoder reads query + document together → more accurate, slow → used for **reranking** top results.

Q: Cosine similarity vs dot product? Cosine ignores magnitude; equals dot product on normalized vectors. Use what the embedding model was trained with.

4. Transformers & pretrained models

(Attention mechanics in 08 §4.)

Q: BERT vs GPT? | BERT | GPT | |---|---| | Encoder-only, bidirectional | Decoder-only, left-to-right (causal mask) | | Pretrained with masked language modeling (MLM) (+NSP originally) | Pretrained with next-token prediction | | Best for understanding: classification, NER, embeddings | Best for generation, general-purpose LLMs |

Q: T5? Encoder-decoder, every task as text-to-text.

Q: Why did Transformers replace RNNs? Parallel training, better long-range dependencies, scale well with data and compute.

Q: What is the context window and what limits it? Max tokens the model can attend to. Limited by $O(n^2)$ attention cost, KV-cache memory, and positional encoding trained lengths (extended via RoPE scaling).

Q: KV cache? During generation, store Keys and Values of previous tokens so each new token only computes attention for itself → much faster decoding, but memory grows with sequence length × batch.

Q: Mixture of Experts (MoE)? Many expert FFNs, a router activates only a few per token → huge parameter count with lower compute per token. Challenges: load balancing, memory.

5. How LLMs are trained

```
1. Pretraining      — next-token prediction on trillions of tokens (web, books, code). Learns
language + knowledge.
2. SFT (instruction tuning) — fine-tune on (instruction, good response) pairs. Learns to follow
instructions/chat format.
3. Preference alignment     — RLHF (reward model + PPO) or DPO (direct preference optimization)
on (chosen, rejected) pairs.
                              Learns helpful, harmless, honest behavior.
(+ reasoning training with RL on verifiable rewards — math/code — for "thinking" models)
```

Q: RLHF in simple words? Humans compare two answers → train a reward model to predict preference → optimize the LLM with RL (PPO) to maximize reward, with a KL penalty to stay close to the original model (avoid reward hacking).

Q: DPO vs RLHF? DPO skips the reward model and RL loop; directly optimizes the model on preference pairs with a classification-like loss. Simpler, more stable.

Q: Scaling laws? Loss decreases predictably as a power law with model size, data, and compute. Chinchilla: for a compute budget, train smaller models on more data (~20 tokens per parameter) than earlier practice.

Q: Emergent abilities / in-context learning? Large models can learn a task from examples in the prompt without weight updates.

6. Using LLMs

Q: Decoding strategies? | Method | Behavior | |---|---| | Greedy | Always pick top token — deterministic, can be repetitive | | Beam search | Keep top-k sequences — good for translation, bland for chat | | Temperature | <1 sharper (more deterministic), >1 more random | | Top-k | sample from k most likely tokens | | Top-p (nucleus) | sample from smallest set with cumulative prob ≥ p | | Repetition/frequency penalty | discourage repeats |

Q: Prompting techniques? Clear instructions + role, few-shot examples, chain-of-thought / reasoning, output format specification (JSON schema), delimiters for untrusted input, self-consistency (sample several, vote), decomposition, ReAct (reason + tool use).

Q: Fine-tuning methods? - **Full fine-tuning:** all weights; expensive, risk of forgetting. - **PEFT:** LoRA (low-rank adapters on attention/MLP weights — trains <1% params), **QLoRA** (LoRA on 4-bit quantized base → fine-tune big models on one GPU), prefix/prompt tuning, adapters. - **Q: How does LoRA work?** Freeze W; learn $\Delta W = B \cdot A$ with small rank r. At inference merge or swap adapters. Hyperparameters: rank r, alpha, target modules, LR.

Q: When fine-tune vs RAG vs prompt? See 02 §12. One-liner: "*RAG for knowledge, fine-tuning for behavior/format/cost, prompting first always.*"

Q: Structured output? JSON mode / function calling / constrained decoding (grammar) + schema validation + retry.

Q: Small vs large models? Small: cheaper, faster, can be fine-tuned and self-hosted, good for narrow tasks (classification, extraction). Large: better reasoning, generalization, fewer examples needed.

7. LLM limitations & evaluation

Q: Why do LLMs hallucinate? They're trained to produce plausible next tokens, not verified truths; knowledge gaps/cutoffs; training rewarded confident answers; no grounding. Mitigate: RAG with citations, "I don't know" option, lower temperature, verification step, tool use (calculator, search).

Q: Other limitations? Knowledge cutoff, context limits, lost-in-the-middle, sensitivity to prompt wording, weak arithmetic without tools, bias, prompt injection, cost/latency, non-determinism.

Q: How to evaluate LLMs? - Benchmarks (MMLU, GSM8K, HumanEval…) — general capability, beware contamination. - **Task-specific eval set** from real use — the most important. - Metrics: exact match / F1 (QA), BLEU/ROUGE (translation/summarization — weak correlation with quality), BERTScore, pass@k (code), LLM-as-judge with rubrics, human eval, pairwise preference (win rate). - Safety/red-teaming, consistency, latency, cost.

8. What-if scenarios

Your text classifier works on English but fails on Hinglish / code-mixed text.

Data mismatch. Collect & label code-mixed samples, use multilingual model (XLM-R, mBERT, multilingual LLM), transliteration normalization, augmentation via translation, evaluate per language.

Sentiment model misclassifies sarcasm and negation ("not bad at all").

BoW/TF-IDF models ignore order → use contextual models (fine-tuned Transformer); add targeted training examples; error analysis with slices (negation, sarcasm).

Fine-tuned LLM outputs repeat the same phrase endlessly.

Decoding: add repetition penalty, adjust temperature/top-p, set max tokens, stop sequences. Training: check EOS token included in training data (common bug — model never learned to stop!), data has duplicates, chat template mismatch.

Fine-tuned model ignores the instruction format after training.

Chat template mismatch between training and inference (system/user/assistant tags), loss computed on prompt tokens instead of only response tokens, too few epochs or LR too low.

LLM answers well in testing, but users report it is "dumb" for some queries.

Collect failing examples, categorize (missing knowledge → RAG; reasoning → bigger model / CoT; format → prompt/fine-tune; ambiguity → clarifying questions). Build eval set from real failures.

NER model misses new product names.

Out-of-distribution entities. Gazetteers/dictionaries, active learning on new data, regular retraining, LLM-based extraction with few-shot examples, character/subword features.

Embedding search returns semantically similar but wrong results ("cancel subscription" matches "subscribe").

Embeddings capture topic not intent/negation. Use hybrid with keyword, fine-tune embeddings with hard negatives, add a cross-encoder reranker, intent classifier.

Summarization model produces facts not in the source.

Abstractive hallucination. Faithfulness checks (NLI entailment, QA-based), constrain with extractive-first, lower temperature, fine-tune on faithful data, citations.

Context window is too small for a 300-page document.

Chunk + map-reduce summarization, RAG, hierarchical summarization, long-context model (trade-off: cost, lost-in-the-middle).

LLM gives different answer quality for the same prompt in different languages.

Tokenization inefficiency + less training data in that language. Translate-in/translate-out pipeline, multilingual eval, language-specific few-shot examples.

Model output contains toxic or biased content.

Input/output safety classifiers, system prompt policies, preference tuning with safety data, red-teaming, human review loops, logging and incident process.

Training an LLM: loss stops improving early.

Data quality/duplication, LR schedule, tokenizer issues, too-small model for data (capacity), check for data loading bugs (same shard repeated), monitor gradient norm.

10 — Computer Vision

Contents

1. Core tasks
2. Key concepts & models
3. Metrics
4. Data & augmentation
5. Deployment
6. What-if scenarios (CV)

---

1. Core tasks

Task	Output	Typical models
Classification	one label per image	ResNet, EfficientNet, ViT, ConvNeXt
Object detection	boxes + labels	YOLO family, Faster R-CNN, DETR, RT-DETR
Semantic segmentation	class per pixel	U-Net, DeepLab, SegFormer
Instance segmentation	mask per object	Mask R-CNN, SAM (promptable)
Keypoints / pose	body/face points	HRNet, OpenPose
OCR	text from image	detection + recognition (CRNN), TrOCR, document AI models
Image similarity / retrieval	embedding	CLIP, DINOv2, metric learning
Generation	images	Diffusion models
Vision-language	captions, VQA	CLIP, multimodal LLMs
Video	actions, tracking	3D CNN, video transformers, SORT/DeepSORT/ByteTrack

2. Key concepts & models

Q: One-stage vs two-stage detectors? Two-stage (Faster R-CNN): region proposals then classify → more accurate, slower. One-stage (YOLO, SSD, RetinaNet): predict boxes directly → fast, real-time.

Q: Anchor boxes? Predefined box shapes the model adjusts. Anchor-free detectors (FCOS, newer YOLOs, DETR) predict directly.

Q: Non-Maximum Suppression (NMS)? Remove duplicate boxes: keep highest-score box, drop others with IoU above threshold. Too low threshold → misses overlapping objects (crowds).

Q: IoU? Intersection area / union area of predicted vs ground-truth box.

Q: U-Net? Encoder-decoder with skip connections that keep fine spatial detail — standard for segmentation (especially medical).

Q: Vision Transformer (ViT)? Split image into patches, embed them as tokens, apply Transformer. Needs lots of data or pretraining; less built-in inductive bias than CNNs.

Q: CLIP? Image encoder + text encoder trained contrastively on image-text pairs → shared embedding space → zero-shot classification and image search with text.

Q: Transfer learning in CV? Start from ImageNet/foundation-model weights; replace head; fine-tune. Huge gains with small data.

Q: Focal loss? Down-weights easy examples, focuses on hard ones — fixes foreground/background imbalance in detection.

Q: Self-supervised learning in vision? SimCLR/MoCo (contrastive), MAE (masked patches), DINO — learn features without labels.

3. Metrics

- Classification: accuracy, precision/recall/F1, top-5 accuracy, confusion matrix.
- Detection: **mAP** (mean Average Precision across classes) at IoU 0.5 or averaged over 0.5:0.95 (COCO).
- Segmentation: mIoU, Dice coefficient.
- Generation: FID, CLIP score, human eval.
- Production: latency (ms/frame), FPS, model size.

4. Data & augmentation

- Augmentations: flips, crops, rotation, color jitter, blur, noise, cutout, mixup, cutmix, mosaic (YOLO). **Must be label-preserving** (don't flip text/digits; for detection, transform boxes too).
- Labeling: bounding boxes / polygons tools (CVAT, Label Studio), label QA, active learning.
- Synthetic data for rare cases.
- Normalize with the same mean/std used in pretraining.

5. Deployment

- Edge (mobile, cameras): quantization (INT8), pruning, distillation, TensorRT/ONNX/CoreML/TFLite, smaller backbones (MobileNet, EfficientNet-Lite).
- Video pipelines: decode → sample frames (not every frame) → batch inference → tracking to smooth → events.
- Trade-off: accuracy vs FPS vs power.

6. What-if scenarios

Model works in the lab but fails on real camera images.

Domain shift: lighting, angle, blur, resolution, compression, camera sensor. Collect production samples, label, fine-tune; augmentation matching real conditions; check preprocessing (RGB vs BGR, resize method,

normalization).

Detector misses small objects.

Higher input resolution, feature pyramid (FPN), tiling/slicing large images (SAHI), anchor sizes tuned, more small-object training examples.

Too many false positives at night.

Night data under-represented → collect/augment low-light data, separate threshold or model for night, IR-aware preprocessing, hard negative mining.

Model is accurate but too slow for real-time on edge device.

Smaller backbone, quantization, pruning, distillation, lower resolution, TensorRT, skip frames + tracking between detections.

Model learned the background instead of the object ("cows on grass" problem).

Spurious correlation. Use Grad-CAM to see where the model looks, diversify backgrounds, augmentation (cutout, background replacement), counterfactual examples.

Medical images: only 200 labeled scans.

Transfer learning (from domain-specific pretrained models), heavy augmentation, self-supervised pretraining on unlabeled scans, cross-validation, expert-in-the-loop active learning, careful patient-level splits (no same patient in train & test!).

Overlapping objects in crowds are missed.

NMS threshold too aggressive → tune, Soft-NMS, or NMS-free detectors (DETR).

Accuracy dropped after a camera firmware update.

Input distribution changed (color profile, compression). Monitor input statistics (brightness, blur), detect drift, recalibrate/fine-tune.

OCR fails on handwritten or rotated text.

Deskew/rotation correction, specific handwriting models, fine-tune on domain data, multimodal LLM fallback with confidence checks.

11 — Statistics, Probability & A/B Testing

Asked in ML engineer, data scientist and applied scientist interviews.

Contents

1. Probability
2. Distributions
3. Statistics & inference
4. A/B testing
5. Classic puzzle questions
6. What-if scenarios

---

1. Probability

- **Conditional:** $P(A|B) = P(A \cap B) / P(B)$.
- **Independence:** $P(A \cap B) = P(A) \cdot P(B)$.
- **Bayes' theorem:** $P(A|B) = P(B|A) \cdot P(A) / P(B)$.
- **Expectation** is linear: $E[X+Y] = E[X] + E[Y]$ (always, even if dependent).
- **Variance:** $Var(X) = E[X^2] - E[X]^2$. $Var(aX) = a^2 Var(X)$.

Classic Bayes question: Disease affects 1% of people. Test is 99% sensitive, 95% specific (5% false positive). You test positive — chance you're sick? $P(sick|+) = (0.99 \cdot 0.01) / (0.99 \cdot 0.01 + 0.05 \cdot 0.99) = 0.0099 / 0.0594 \approx$ **16.7%**. Lesson: low base rates → most positives are false (same reason fraud alerts are mostly false).

2. Distributions

Distribution	Use
Bernoulli / Binomial	yes/no events, number of successes in n trials (clicks)
Poisson	counts per time interval (requests/minute)
Normal	sums/averages (Central Limit Theorem)
Exponential	time between events
Uniform	equal likelihood
Power law / long tail	popularity, word frequency

Central Limit Theorem: the mean of many independent samples is approximately normal, whatever the original distribution → why t-tests/z-tests work for averages.

3. Statistics & inference

Q: p-value? Probability of observing results at least this extreme **if the null hypothesis were true**. It is NOT the probability the null is true.

Q: Type I vs Type II error? Type I: false positive (reject a true null), rate = α (usually 0.05). Type II: false negative (miss a real effect), rate = β. **Power** = 1 − β (usually 0.8).

Q: Confidence interval? If we repeated the experiment many times, 95% of such intervals would contain the true value.

Q: Which test? t-test (compare means), z-test for proportions (conversion rates), chi-square (categorical independence), Mann-Whitney (non-normal), bootstrap (anything, by resampling).

Q: Correlation vs causation? Correlation can come from confounders or reverse causation. Causation needs randomized experiments or causal methods (diff-in-diff, instrumental variables, propensity matching).

Q: Simpson's paradox? A trend in each group reverses when groups are combined, due to a confounding variable (e.g., different group sizes). Always check segment-level results.

Q: MLE vs MAP? MLE maximizes likelihood of data. MAP adds a prior (= regularization: Gaussian prior ↔ L2, Laplace prior ↔ L1).

4. A/B testing

Steps: 1. Hypothesis + **primary metric** (e.g., conversion) + **guardrail metrics** (latency, revenue, errors). 2. Randomization unit (user, not request — avoid same user seeing both). 3. **Sample size** via power analysis (baseline rate, minimum detectable effect, α, power). 4. Run for full weeks (weekday/weekend cycles); **don't peek** and stop early when significant. 5. Analyze: significance, confidence intervals, segments; check sanity (sample ratio mismatch). 6. Decide considering practical significance, not only statistical.

Pitfalls to mention: - **Peeking** → inflated false positives (use sequential testing if you must). - **Multiple comparisons** → many metrics/variants → Bonferroni or FDR correction. - **Novelty / primacy effects** → run longer. - **Network effects** (social, marketplaces) → cluster randomization. - **Sample Ratio Mismatch (SRM):** expected 50/50 but got 52/48 → bug in assignment; don't trust results. - Long-term vs short-term effects → holdout groups.

For ML models: offline evaluation → shadow → A/B test. Interleaving for ranking (faster, more sensitive).

5. Classic puzzle questions

- **Expected number of coin flips to get heads?** 2 (geometric, 1/p).
- **Birthday problem:** 23 people → >50% chance two share a birthday.
- **Monty Hall:** always switch — 2/3 win chance.
- **Two children, at least one boy — probability both boys?** 1/3.
- **Sample from a stream of unknown length uniformly?** Reservoir sampling: keep the i-th item with probability 1/i.

- **Generate a fair coin from a biased one?** Flip twice: HT → H, TH → T, else repeat.

6. What-if scenarios

A/B test shows +5% conversion, but the effect disappears after launch.

Novelty effect, peeking/false positive, seasonality, SRM bug, segment mix changed. Use holdouts, longer tests, pre-registered analysis.

Test result is significant for the overall population but negative for mobile users.

Simpson's paradox or genuine heterogeneity. Investigate segments; consider targeted rollout; be careful of multiple-comparison false findings.

You don't have enough traffic for an A/B test.

Increase MDE, use variance reduction (CUPED — use pre-experiment metric as covariate), run longer, use more sensitive metrics, interleaving, or Bayesian approaches.

Metric moved, but you suspect it's not from your change.

Check A/A test, seasonality, other concurrent experiments, logging changes, external events.

12 — ML Coding Round (Implement From Scratch)

Interviewers ask you to code ML pieces using only NumPy (or PyTorch) in 30–45 minutes. **Practice:** type each one from memory until you can do it in < 15 minutes. Talk while coding: shapes, edge cases, complexity.

Contents

1. Linear regression (gradient descent)
2. Logistic regression
3. K-means
4. k-Nearest Neighbors
5. Metrics: precision, recall, F1, AUC
6. Softmax & cross-entropy (stable)
7. Scaled dot-product & multi-head attention
8. A 2-layer neural network with backprop
9. Decision tree split (Gini)
10. PyTorch training loop
11. Cosine similarity top-k search
12. BPE tokenizer merge step
13. Interview tips for coding rounds

---

1. Linear regression

```
import numpy as np

class LinearRegression:
    def __init__(self, lr=0.01, epochs=1000):
        self.lr, self.epochs = lr, epochs

    def fit(self, X, y):                        # X: (n, d), y: (n,)
        n, d = X.shape
        self.w, self.b = np.zeros(d), 0.0
        for _ in range(self.epochs):
            y_hat = X @ self.w + self.b
            err = y_hat - y                     # (n,)
            dw = (2 / n) * X.T @ err            # gradient of MSE
            db = (2 / n) * err.sum()
            self.w -= self.lr * dw
            self.b -= self.lr * db
        return self

    def predict(self, X):
        return X @ self.w + self.b

# Closed form (normal equation): w = (XᵀX)⁻¹ Xᵀy  → O(d³), use np.linalg.lstsq for stability
```

2. Logistic regression

```python
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))   # clip avoids overflow

class LogisticRegression:
    def __init__(self, lr=0.1, epochs=1000, l2=0.0):
        self.lr, self.epochs, self.l2 = lr, epochs, l2

    def fit(self, X, y):
        n, d = X.shape
        self.w, self.b = np.zeros(d), 0.0
        for _ in range(self.epochs):
            p = sigmoid(X @ self.w + self.b)
            dw = X.T @ (p - y) / n + self.l2 * self.w   # gradient of log loss (+ L2)
            db = (p - y).mean()
            self.w -= self.lr * dw
            self.b -= self.lr * db
        return self

    def predict_proba(self, X):
        return sigmoid(X @ self.w + self.b)

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)

def log_loss(y, p, eps=1e-12):
    p = np.clip(p, eps, 1 - eps)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))
```

3. K-means

```python
def kmeans(X, k, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    centroids = X[rng.choice(len(X), k, replace=False)]
    for _ in range(iters):
        # distances: (n, k)
        dists = np.linalg.norm(X[:, None, :] - centroids[None, :, :], axis=2)
        labels = dists.argmin(axis=1)
        new = np.array([X[labels == j].mean(axis=0) if np.any(labels == j) else centroids[j]
                        for j in range(k)])               # keep old centroid if cluster empty
        if np.allclose(new, centroids):
            break
        centroids = new
    return centroids, labels
# Mention: k-means++ init, scaling features, O(n·k·d) per iteration.
```

4. k-Nearest Neighbors

```python
from collections import Counter

def knn_predict(X_train, y_train, X_test, k=3):
    preds = []
    for x in X_test:
```

```
        d = np.linalg.norm(X_train - x, axis=1)
        idx = np.argpartition(d, k)[:k]          # O(n) instead of full sort
        preds.append(Counter(y_train[idx]).most_common(1)[0][0])
    return np.array(preds)
```

5. Metrics

```
def precision_recall_f1(y_true, y_pred):
    tp = np.sum((y_true == 1) & (y_pred == 1))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
    return precision, recall, f1

def roc_auc(y_true, scores):
    # AUC = probability a random positive scores higher than a random negative
    order = np.argsort(scores)
    ranks = np.empty(len(scores)); ranks[order] = np.arange(1, len(scores) + 1)
    n_pos = y_true.sum(); n_neg = len(y_true) - n_pos
    return (ranks[y_true == 1].sum() - n_pos * (n_pos + 1) / 2) / (n_pos * n_neg)
    # (ties need average ranks — mention it)
```

6. Softmax & cross-entropy

```
def softmax(z, axis=-1):
    z = z - z.max(axis=axis, keepdims=True)    # numerical stability: subtract max
    e = np.exp(z)
    return e / e.sum(axis=axis, keepdims=True)

def cross_entropy(logits, labels):             # logits (n, C), labels (n,) ints
    z = logits - logits.max(axis=1, keepdims=True)
    log_probs = z - np.log(np.exp(z).sum(axis=1, keepdims=True))   # log-softmax
    return -log_probs[np.arange(len(labels)), labels].mean()
# Gradient w.r.t. logits = softmax(logits) - one_hot(labels), divided by n
```

7. Attention

```
def scaled_dot_product_attention(Q, K, V, mask=None):
    # Q: (..., Lq, d), K: (..., Lk, d), V: (..., Lk, dv)
    d = Q.shape[-1]
    scores = Q @ K.swapaxes(-1, -2) / np.sqrt(d)       # (..., Lq, Lk)
    if mask is not None:
        scores = np.where(mask, scores, -1e9)          # mask=False → blocked
    weights = softmax(scores, axis=-1)
    return weights @ V, weights

def causal_mask(L):
    return np.tril(np.ones((L, L), dtype=bool))        # token i sees tokens ≤ i
```

```python
import torch, torch.nn as nn, torch.nn.functional as F

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        assert d_model % n_heads == 0
        self.h, self.dk = n_heads, d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model)
        self.out = nn.Linear(d_model, d_model)

    def forward(self, x, causal=False):                # x: (B, L, D)
        B, L, D = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        # (B, L, D) -> (B, h, L, dk)
        q, k, v = [t.view(B, L, self.h, self.dk).transpose(1, 2) for t in (q, k, v)]
        scores = q @ k.transpose(-2, -1) / self.dk ** 0.5          # (B, h, L, L)
        if causal:
            mask = torch.triu(torch.ones(L, L, dtype=torch.bool, device=x.device), 1)
            scores = scores.masked_fill(mask, float('-inf'))
        attn = F.softmax(scores, dim=-1)
        y = (attn @ v).transpose(1, 2).contiguous().view(B, L, D)
        return self.out(y)
```

8. Two-layer neural network

```python
class TwoLayerNet:
    def __init__(self, d_in, d_hidden, n_classes, lr=0.1, seed=0):
        rng = np.random.default_rng(seed)
        self.W1 = rng.normal(0, np.sqrt(2 / d_in), (d_in, d_hidden))   # He init
        self.b1 = np.zeros(d_hidden)
        self.W2 = rng.normal(0, np.sqrt(2 / d_hidden), (d_hidden, n_classes))
        self.b2 = np.zeros(n_classes)
        self.lr = lr

    def step(self, X, y):                                  # one gradient step, returns loss
        n = len(X)
        # forward
        h_pre = X @ self.W1 + self.b1
        h = np.maximum(0, h_pre)                           # ReLU
        logits = h @ self.W2 + self.b2
        probs = softmax(logits)
        loss = -np.log(probs[np.arange(n), y] + 1e-12).mean()
        # backward
        dlogits = probs.copy(); dlogits[np.arange(n), y] -= 1; dlogits /= n
        dW2 = h.T @ dlogits;      db2 = dlogits.sum(0)
        dh = dlogits @ self.W2.T
        dh_pre = dh * (h_pre > 0)                          # ReLU gradient
        dW1 = X.T @ dh_pre;       db1 = dh_pre.sum(0)
        # update
        for p, g in ((self.W1, dW1), (self.b1, db1), (self.W2, dW2), (self.b2, db2)):
            p -= self.lr * g
        return loss
```

9. Decision tree split

```
def gini(y):
    _, counts = np.unique(y, return_counts=True)
    p = counts / counts.sum()
    return 1 - np.sum(p ** 2)

def best_split(X, y):
    best = (None, None, np.inf)                          # (feature, threshold, weighted gini)
    n = len(y)
    for f in range(X.shape[1]):
        for t in np.unique(X[:, f]):
            left = X[:, f] <= t
            if left.all() or (~left).all():
                continue
            g = (left.sum() * gini(y[left]) + (~left).sum() * gini(y[~left])) / n
            if g < best[2]:
                best = (f, t, g)
    return best
```

10. PyTorch training loop

```
def train(model, train_loader, val_loader, epochs=10, lr=3e-4, device='cuda'):
    model.to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=0.01)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
    loss_fn = nn.CrossEntropyLoss()
    best_val = float('inf')
    for epoch in range(epochs):
        model.train()
        for X, y in train_loader:
            X, y = X.to(device), y.to(device)
            opt.zero_grad()
            loss = loss_fn(model(X), y)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
        sched.step()

        model.eval()                                     # dropout/BN off
        val_loss, n = 0.0, 0
        with torch.no_grad():                            # no graph → saves memory
            for X, y in val_loader:
                X, y = X.to(device), y.to(device)
                val_loss += loss_fn(model(X), y).item() * len(y); n += len(y)
        val_loss /= n
        if val_loss < best_val:
            best_val = val_loss
            torch.save(model.state_dict(), 'best.pt')    # checkpoint best model
        print(f'epoch {epoch}: val_loss={val_loss:.4f}')
```

11. Cosine similarity top-k

```
def top_k_cosine(query, matrix, k=5):          # query (d,), matrix (n, d)
    q = query / np.linalg.norm(query)
    M = matrix / np.linalg.norm(matrix, axis=1, keepdims=True)
    sims = M @ q
    idx = np.argpartition(-sims, k)[:k]
    idx = idx[np.argsort(-sims[idx])]          # sort the top-k only
    return idx, sims[idx]
# Brute force O(n·d). At scale: ANN index (HNSW/IVF) — see 02 §7.
```

12. BPE merge step

```
from collections import Counter

def bpe_train(words, num_merges):
    # words: list of strings; represent each as tuple of symbols
    vocab = Counter(tuple(w) + ('</w>',) for w in words)
    merges = []
    for _ in range(num_merges):
        pairs = Counter()
        for word, freq in vocab.items():
            for a, b in zip(word, word[1:]):
                pairs[(a, b)] += freq
        if not pairs:
            break
        best = max(pairs, key=pairs.get)
        merges.append(best)
        new_vocab = Counter()
        for word, freq in vocab.items():
            out, i = [], 0
            while i < len(word):
                if i < len(word) - 1 and (word[i], word[i + 1]) == best:
                    out.append(word[i] + word[i + 1]); i += 2
                else:
                    out.append(word[i]); i += 1
            new_vocab[tuple(out)] += freq
        vocab = new_vocab
    return merges
```

13. Tips

- Start by stating input/output shapes and assumptions. Write a tiny test example.
- Vectorize with NumPy; mention complexity (time & memory).
- Numerical stability: subtract max in softmax, clip in log/sigmoid, add epsilon.
- Edge cases: empty clusters, division by zero, ties, k > n.
- Other common asks: implement **dropout, batch norm forward, conv2d (naive loops), IoU + NMS, train/test split, stratified sampling, gradient check, beam search, top-p sampling, reservoir sampling, an LRU cache**.

Bonus: IoU + NMS (CV interviews)

```python
def iou(a, b):                                  # boxes [x1, y1, x2, y2]
    x1, y1 = max(a[0], b[0]), max(a[1], b[1])
    x2, y2 = min(a[2], b[2]), min(a[3], b[3])
    inter = max(0, x2 - x1) * max(0, y2 - y1)
    area = lambda r: (r[2] - r[0]) * (r[3] - r[1])
    return inter / (area(a) + area(b) - inter + 1e-9)

def nms(boxes, scores, thresh=0.5):
    order = np.argsort(scores)[::-1]
    keep = []
    while len(order):
        i = order[0]; keep.append(i)
        order = np.array([j for j in order[1:] if iou(boxes[i], boxes[j]) < thresh])
    return keep
```

Bonus: Top-p (nucleus) sampling

```python
def top_p_sample(logits, p=0.9, temperature=1.0, rng=np.random.default_rng()):
    probs = softmax(logits / temperature)
    order = np.argsort(probs)[::-1]
    cum = np.cumsum(probs[order])
    cutoff = np.searchsorted(cum, p) + 1         # smallest set with mass ≥ p
    keep = order[:cutoff]
    kept = probs[keep] / probs[keep].sum()
    return rng.choice(keep, p=kept)
```

13 — MLOps & Production ML (Q&A)

Complements 02-ml-and-llm-systems.md. Asked heavily in ML Engineer / MLOps / AI Engineer interviews.

Core Q&A

Q: What is MLOps? Practices to reliably build, deploy, monitor and improve ML models in production: CI/CD for code **and** data **and** models, reproducibility, automation, monitoring.

Q: How is ML CI/CD different from software CI/CD? Software: test code → deploy. ML adds: data validation, training pipeline, model evaluation gates (must beat current model), model registry, continuous training (CT), and monitoring of data/model quality — not just uptime.

Q: MLOps maturity levels? Level 0: manual notebooks. Level 1: automated training pipeline + continuous training. Level 2: full CI/CD of pipelines with automated testing, deployment and monitoring.

Q: What should you version? Code (git), data (DVC/lakeFS/Delta), features, model artifacts (registry), hyperparameters/config, environment (Docker image, requirements lock), prompts (for LLMs), eval datasets.

Q: Tools you should know by name? | Need | Tools | |---|---| | Orchestration | Airflow, Kubeflow Pipelines, Prefect, Dagster, SageMaker/Vertex Pipelines | | Experiment tracking | MLflow, Weights & Biases, Neptune | | Feature store | Feast, Tecton, Databricks/Vertex/SageMaker feature stores | | Model registry | MLflow Registry, SageMaker/Vertex registries | | Serving | Triton, TorchServe, BentoML, KServe, Ray Serve, vLLM/TGI (LLMs) | | Containers | Docker, Kubernetes (autoscaling, GPU scheduling) | | Monitoring | Prometheus + Grafana, Evidently, Arize, WhyLabs; LLM: Langfuse, LangSmith, Phoenix | | Data quality | Great Expectations, dbt tests, Pandera |

Q: How do you test ML systems? - Unit tests for feature code and preprocessing. - Data tests: schema, ranges, nulls, distribution. - Model tests: performance thresholds on holdout, slice tests (no segment worse than X), invariance tests (changing a name shouldn't change sentiment), directional tests, regression tests on known hard cases. - Integration tests: end-to-end pipeline on sample data. - Load tests: latency/throughput of serving.

Q: Deployment strategies? Shadow → canary → A/B → full; blue-green; keep previous version for instant rollback. See 02 §5.

Q: Online vs batch inference trade-offs? See 02 §4.

Q: How to make inference cheaper/faster? Batching, quantization, distillation, pruning, ONNX/TensorRT compilation, caching predictions, right-sizing hardware (CPU often enough for small models), autoscaling, spot instances for batch.

Q: How do you containerize and scale a model? Docker image with model + server, health/readiness endpoints, deploy on Kubernetes with HPA (autoscale on CPU/GPU/QPS/queue length), resource limits, GPU node pools, model loaded at startup (or pulled from registry).

Q: Continuous training triggers? Schedule, new labeled data volume, drift detected, performance drop.

Q: What is training-serving skew and how to prevent it? See 04 #23: shared feature code/feature store, log serving features and use them for training, compare distributions.

Q: Feedback loops? Model's predictions affect future training data (recommenders only learn from items shown). Mitigate with exploration, randomized holdouts, logging propensities.

Q: Responsible AI in production? Fairness checks by slice, explainability, model cards, data privacy (PII handling, retention), audit trails, human oversight for high-stakes decisions.

Q: LLMOps — what's different? Prompt versioning, eval suites with LLM judges, token cost monitoring, tracing multi-step chains/agents, guardrails, provider fallback, managing model version changes from vendors, RAG index pipelines.

What-if scenarios (MLOps)

- **Training pipeline fails at 3 AM** → alerting, retries for transient errors, idempotent steps, checkpointing; keep serving the previous model (never block serving on training).
- **Model registry has a model but no one knows what data it used** → enforce lineage metadata on registration (data version, code commit, metrics) in the pipeline; block manual uploads.
- **Kubernetes pods with the model restart constantly (OOMKilled)** → memory limits too low for model + batch, memory leak, too many workers each loading the model; right-size, share model across workers, smaller batch.
- **Cold starts take 2 minutes** → bake model in image or fast storage, keep min replicas, lazy-load less used models, smaller model formats.
- **Multiple teams deploy models differently, causing chaos** → internal ML platform: templates, standard serving, registry, monitoring defaults (paved road).
- **Cloud bill doubled** → find idle GPUs, oversized instances, unused endpoints, untagged experiments; autoscale to zero, spot/preemptible for training, budgets & alerts per team.

14 — DSA Coding Round (for AI/ML roles)

Most ML/AI engineer loops include 1–2 standard coding rounds (LeetCode-style, usually medium). You don't need 500 problems — you need the **~15 patterns**. Learn the pattern, then do 5–8 problems each.

The method for every coding question (say it out loud)

1. **Clarify:** input size, types, edge cases (empty, duplicates, negatives), return format.
2. **Example:** walk through a small example by hand.
3. **Brute force first:** state it and its complexity.
4. **Optimize:** which pattern fits? State new complexity.
5. **Code** cleanly with good names.
6. **Test:** run your example + edge cases by hand.
7. **Complexity:** time and space.

Patterns → when to use → classic problems

#	Pattern	Signal in the question	Practice problems
1	**Hash map / set**	"find pair", "count", "seen before", O(1) lookup	Two Sum, Group Anagrams, Longest Consecutive Sequence
2	**Two pointers**	sorted array, pairs, palindrome, in-place	3Sum, Container With Most Water, Valid Palindrome
3	**Sliding window**	contiguous subarray/substring, "longest/shortest with condition"	Longest Substring Without Repeating, Minimum Window Substring
4	**Prefix sum**	range sums, "subarray sum equals k"	Subarray Sum Equals K, Range Sum Query
5	**Binary search**	sorted data, "minimum X such that…", search on answer	Search in Rotated Array, Koko Eating Bananas, Median of Two Sorted Arrays
6	**Stack / monotonic stack**	matching brackets, "next greater element"	Valid Parentheses, Daily Temperatures, Largest Rectangle in Histogram
7	**Heap / priority queue**	top-k, k-th largest, merge sorted, streaming median	Kth Largest, Top K Frequent, Merge K Sorted Lists, Find Median from Data Stream
8	**Linked list**	reverse, cycle, merge	Reverse List, Linked List Cycle, LRU Cache (hash map + doubly linked list)
9	**Trees (DFS/BFS)**	hierarchy, depth, paths	Max Depth, Level Order, Validate BST, Lowest Common Ancestor, Serialize Tree

#	Pattern	Signal in the question	Practice problems
10	**Graphs (BFS/DFS/union-find/topological sort)**	grid, connections, dependencies, shortest path	Number of Islands, Course Schedule, Clone Graph, Dijkstra (Network Delay Time)
11	**Backtracking**	"all combinations/permutations/subsets"	Subsets, Permutations, Combination Sum, Word Search
12	**Dynamic programming**	"number of ways", "min/max", overlapping subproblems	Climbing Stairs, Coin Change, Longest Increasing Subsequence, Edit Distance, LCS
13	**Greedy / intervals**	scheduling, merge intervals	Merge Intervals, Meeting Rooms II, Jump Game
14	**Trie**	prefix search, autocomplete	Implement Trie, Word Search II
15	**Bit manipulation / math**	XOR tricks, powers	Single Number, Counting Bits

ML-flavored coding questions (common at AI companies): Edit distance (NLP), LRU cache (model cache), top-k frequent words, reservoir sampling, matrix ops, implement tokenizer, rate limiter, moving average from a stream, merge k sorted lists (merge search results), autocomplete with trie, sample from a weighted distribution (prefix sums + binary search).

Complexity cheat sheet

Operation	Complexity
Hash map get/put	O(1) avg
Sort	O(n log n)
Binary search	O(log n)
Heap push/pop	O(log n); build heap O(n)
BFS/DFS	O(V + E)
Top-k with heap	O(n log k)
Recursion space	O(depth)

Rough limits: $n \le 10^5$ → need O(n log n) or better. $n \le 10^3$ → $O(n^2)$ ok. $n \le 20$ → exponential/backtracking ok.

Python templates

```python
# Sliding window (variable size)
def longest_unique_substring(s):
    seen, left, best = {}, 0, 0
    for right, ch in enumerate(s):
        if ch in seen and seen[ch] >= left:
            left = seen[ch] + 1
        seen[ch] = right
        best = max(best, right - left + 1)
```

```
    return best

# Binary search on answer
def min_feasible(lo, hi, ok):            # smallest x in [lo, hi] with ok(x) True
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid): hi = mid
        else: lo = mid + 1
    return lo

# BFS on grid
from collections import deque
def bfs(grid, start):
    R, C = len(grid), len(grid[0])
    q, seen = deque([start]), {start}
    while q:
        r, c = q.popleft()
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and (nr, nc) not in seen and grid[nr][nc] != '#':
                seen.add((nr, nc)); q.append((nr, nc))
    return seen

# Top-k with heap
import heapq
from collections import Counter
def top_k_frequent(nums, k):
    return [x for x, _ in heapq.nlargest(k, Counter(nums).items(), key=lambda t: t[1])]

# LRU cache
from collections import OrderedDict
class LRUCache:
    def __init__(self, cap): self.cap, self.d = cap, OrderedDict()
    def get(self, k):
        if k not in self.d: return -1
        self.d.move_to_end(k); return self.d[k]
    def put(self, k, v):
        self.d[k] = v; self.d.move_to_end(k)
        if len(self.d) > self.cap: self.d.popitem(last=False)

# DP: edit distance
def edit_distance(a, b):
    dp = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
        prev, dp[0] = dp[0], i
        for j in range(1, len(b) + 1):
            cur = dp[j]
            dp[j] = prev if a[i-1] == b[j-1] else 1 + min(prev, dp[j], dp[j-1])
            prev = cur
    return dp[-1]
```

8-week plan

Weeks	Patterns	Daily
1–2	1–4 (hash, pointers, window, prefix)	2 problems

Weeks	Patterns	Daily
3–4	5–9 (binary search, stack, heap, lists, trees)	2 problems
5–6	10–12 (graphs, backtracking, DP)	2 problems
7–8	13–15 + mixed timed sets	1 timed mock (45 min, 2 problems)
Use a curated list (e.g., "NeetCode 150" / "Blind 75") — it covers these patterns.		

15 — Behavioral & Project Deep-Dive Round

For ML/AI roles, **"Tell me about a project you built"** is often the round that decides the offer. Interviewers dig 5–6 levels deep with "why?" and "what if?". If you can't explain *your own* decisions, it looks like you didn't make them.

Part A — The project deep dive

Prepare 2–3 projects using this template (write it down!)

Section	Your notes
1. Problem & business goal	What problem, for whom, why it mattered, how success was measured (business metric)
2. Your role	What *you* did vs. the team. Use "I".
3. Data	Source, size, labels (how obtained, quality), splits, imbalance, leakage risks you handled
4. Baseline	What simple approach did you compare against? Its score?
5. Approach	Model/architecture and **why this one over alternatives**
6. Evaluation	Offline metrics (why these?), validation strategy, error analysis findings
7. Deployment	How served (batch/real-time), latency, infra, monitoring
8. Results / impact	Numbers: "+12% recall at same precision", "cut cost 40%", "saved 20 hrs/week"
9. Challenges	The hardest problem and how you debugged it (use C.L.H.I.F.P. from 03)
10. Trade-offs	What you sacrificed and why
11. What you'd do differently	Shows maturity
12. Next steps	How you'd improve with more time/data

Questions you WILL be asked — prepare an answer for each

Data - How much data? How was it labeled? How did you check label quality? - Was it imbalanced? How did you handle it? - How did you split the data? Could there be leakage? - What features mattered most? Any surprising ones?

Model - Why this model? What else did you try? Why not a simpler/bigger one? - What hyperparameters mattered? How did you tune them? - Why this loss/metric?

Evaluation - How do you know it actually works? Offline vs online results? - Where does it fail? (error analysis — always have 2–3 failure categories ready) - Did you compare against a baseline?

Production - How is it served? Latency? Cost? - How do you monitor it? What happens when it degrades? - How often retrained?

What-ifs (this is where people fail — practice!) - What if the data doubled / was 100× bigger? - What if labels were unavailable? - What if latency had to be 10× lower? - What if the data distribution changed? - What if you had to explain predictions to a regulator? - What if a user group gets worse results? - (For RAG/LLM projects:) what if answers are inconsistent / hallucinated / slow / expensive? → 04

Golden rule: if you don't know, say *"We didn't measure that, but here's how I would check it: …"* — that still scores well. Never invent numbers.

Example (RAG project) — 2-minute pitch

> "I built an internal Q&A assistant over 50k HR and IT documents for ~5,000 employees. The goal was reducing support tickets. I owned the retrieval pipeline and evaluation. Baseline was keyword search, which answered ~40% of a labeled set of 300 real questions. I used structure-aware chunking with section headers, hybrid BM25 + embedding search, and a cross-encoder reranker. Recall@5 went from 58% to 86%. For generation we used temperature 0 with mandatory citations and an 'I don't know' path when retrieval scores were low; faithfulness on our eval set was 94%. The hardest problem was inconsistent answers — we traced it to duplicate policy versions in the corpus and fixed it with effective-date metadata and dedupe. After launch, IT tickets for covered topics dropped 25%. If I did it again, I'd build the eval set on day one instead of week four."

That pitch answers 10 follow-ups before they're asked.

Part B — Behavioral questions (STAR method)

STAR: Situation (1 line) → **T**ask (your responsibility) → **A**ction (what *you* did — 60% of the answer) → **R**esult (numbers + what you learned).

Prepare 6–8 stories that cover these

Theme	Typical question
Conflict	Tell me about a disagreement with a teammate/manager.
Failure	Tell me about a project that failed / a mistake you made.
Ambiguity	A time requirements were unclear.
Ownership	A time you went beyond your role.
Influence	Convincing others without authority (e.g., "use a simpler model").
Deadline / pressure	Delivering under a tight timeline; what you cut.
Learning	Learning a new technology quickly.
Impact	Your most impactful work.
Stakeholders	Explaining ML to non-technical people.
Ethics / quality	Pushing back on shipping a model that wasn't ready.

One story can cover several themes — map each story to 2–3 themes.

Good vs weak answer — "Tell me about a failure"

- ❌ "I'm a perfectionist…" / blaming others / no lesson.
- ✅ "I shipped a churn model without slice-level evaluation. Two weeks later sales noticed it flagged nearly all new customers — a feature was missing for accounts < 30 days old. I rolled back, added a default-handling fix and slice tests to our pipeline. Since then I always evaluate by segment before launch. The relaunched model hit its target."

Questions to ask the interviewer (always have 2–3)

- What does success look like in this role in the first 6 months?
- How do models/LLM features get from prototype to production here? How do you evaluate them?
- What's the biggest technical challenge the team is facing right now?

Part C — Communication habits that change outcomes

- **Think out loud.** Silence = interviewer can't give you credit.
- **Structure first:** "I'll cover three things: data, model, and deployment."
- **Numbers over adjectives:** "p95 latency 400 ms" beats "pretty fast".
- **Admit unknowns, then reason:** "I haven't used X, but I'd expect it to behave like Y because…"
- **Check in:** "Does this level of detail make sense, or should I go deeper on retrieval?"
- **Simple English is fine.** Clear beats fancy. Short sentences.